

2026年江南大学 第十届校内选拔赛

参赛论文

题目：我国碳排放时空特征分析与趋势预测

学院：光电信息物理与科学学院

学号：1130524220 姓名：胡珩瑄

学号：1130524218 姓名：王觉幸

学号：1130524217 姓名：陈平原

2026 年 5 月 20 日

我国碳排放时空特征分析与趋势预测

摘要

碳达峰、碳中和是我国推进高质量发展和生态文明建设的重要战略目标，也与能源供应、产业升级和居民低碳生活密切相关。不同地区资源禀赋、产业基础和发展阶段差异明显，导致省际碳排放水平和减排压力并不均衡。研究我国碳排放时空特征、影响因素和未来趋势，有助于为区域差异化减排和“双碳”政策实施提供量化依据。

对于问题一，首先基于 2022 年 30 省碳排放清单，选取 CO_2 总量、人均 CO_2 、碳排放强度、煤炭相关排放占比等指标进行统计分析，并结合省级空间分布结果识别高排放集聚区域。其次，设各省低碳综合评价得分为决策变量 S_i ，建立熵权 TOPSIS 综合评价模型；进一步构造 K-means 聚类模型，对省份进行分类分级。模型采用 Python 实现并求解。结果表明，我国省际碳排放呈现“北方资源能源高排放带、东部沿海制造业高排放带、中西南相对低排放区”的格局；山东、内蒙古、河北、江苏、山西、广东、河南、辽宁是排放总量最高的省份。资源依赖高排放型省份主要包括河北、山西、内蒙古、宁夏、新疆，应重点控煤和推动资源型产业转型；北京、上海属于经济发达效率型，可承担技术示范和绿色金融引领功能。

对于问题二，首先围绕能源消费结构、产业结构、人均 GDP、城镇化率和人口规模等因素分析碳排放驱动机制。其次，设碳排放总量 C_i 为因变量，人口规模 P_i 、人均 GDP A_i 、煤炭相关排放占比 Coal_i 、第二产业占比 IS_i 、城镇化率 U_i 为解释变量，建立 STIRPAT 模型，并采用 OLS 估计、VIF 检验和岭回归稳健性检验。结果表明，当前省际碳排放差异最主要由人口规模和煤炭依赖共同驱动，其中煤炭相关排放占比是最关键、最稳定的结构性因素；人均 GDP、第二产业占比和城镇化率的影响更多通过能源结构和产业结构间接体现。因此，单纯压缩经济活动并非合理路径，降低煤炭占比、推进重点行业节能降碳、提升经济产出效率才是控制排放的核心方向。

对于问题三，首先以 2024 年全国实际排放量 11901.44 Mt 为基准，分析 2026–2045 年全国碳排放趋势。其次，将问题二得到的 STIRPAT 模型系数转化为年度递推模型，设置基准、低碳、强化低碳三种情景预测方案，并以人口增长率、人均 GDP 增长率、煤炭占比变化、第二产业占比变化和城镇化率变化作为情景参数。模型采用 Python 递推求解并进行灵敏度分析。结果表明，若维持基准转型速度，全国可能在 2032 年左右达峰，峰值约 12435 Mt，达峰偏晚且峰值较高；低碳情景可将达峰时间提前到 2026 年左右，2045 年排放降至 8903 Mt；强化低碳情景下降幅最大，2045 年排放约 7114 Mt，较峰值下降 40.2%。因此，实现更早达峰和更大减排潜力的关键在于加快煤炭替代、优化产业结构并提高单位 GDP 产出效率。

对于问题四，在前三问结果基础上，本文从区域分类、关键驱动因素和情景预测结论出发提出差异化减排方案。结果表明，资源依赖高排放型省份应以控煤、煤电改造和新能源基地建设为重点；工业制造高排放型省份应优先推进钢铁、建材、电力、石化等行业节能降碳；中等转型压力型省份应严控高耗能产业转移；北京、上海等效率型地区应发挥绿色金融、碳交易和技术示范作用。总体看，降低煤炭占比和提高产业效率是兼顾达峰时间与减排潜力的核心政策抓手。

关键词：熵权 TOPSIS；K-means 聚类；STIRPAT 模型；情景预测

一、问题重述

1.1 问题背景

全球气候变暖已成为人类社会面临的共同挑战，极端天气、能源安全和产业转型压力不断加大，控制温室气体排放已成为各国经济社会发展的重要议题。我国提出碳达峰、碳中和目标，要求在保障经济增长和居民生活质量的同时，持续推进能源结构调整、产业低碳升级和绿色生活方式转变。由于我国不同省份在资源禀赋、人口规模、产业结构、能源结构和经济发展水平方面差异明显，碳排放总量、排放效率和减排压力也表现出显著的空间不均衡性。因此，准确刻画省际碳排放分布特征，识别排放增长的关键驱动因素，并预测未来碳排放变化趋势，对于制定差异化、可考核、可持续的区域减排政策具有重要意义。



图 1 碳排放空间差异与绿色转型背景示意图

1.2 问题提出

为了服务我国区域差异化减排和碳达峰路径研判，现需结合题目附件和相关统计资料建立数学模型，重点分析以下问题：

- **问题一：**选取碳排放总量、人均碳排放、碳排放强度等核心指标，分析 30 个省份碳排放的空间分布特征和差异显著性，并在此基础上建立省份分类分级模型。
- **问题二：**从能源消费结构、产业结构、人均 GDP、城镇化率、人口规模等因素中识别影响碳排放的核心驱动因素，构建定量模型并完成参数估计、显著性检验和模型优化。
- **问题三：**基于前述驱动因素模型，设置基准、低碳、强化低碳三种发展情景，预测 2026–2045 年全国碳排放总量与碳排放强度变化趋势，判断碳达峰时间节点、峰值水平和减排潜力。
- **问题四：**综合省份分类结果、驱动因素识别结果和情景预测结论，提出具有区域针对性和可操作性的减排政策建议。

二、问题分析

2.1 问题一的分析

问题一需要回答两个递进层面。首先，要判断各省碳排放是否存在明显的空间分布特征和差异显著性，这不仅要比较排放总量，还要结合地理位置观察高排放区域是否呈

现集聚现象。其次，要在差异分析基础上对省份进行分类分级，使结果能够服务于后续政策建议。若只比较 CO₂ 总量，容易把人口规模大、经济总量大的省份简单判定为高排放压力省份，而忽视其单位人口排放、单位经济产出排放和能源结构差异。因此，问题一需要同时兼顾排放规模、排放效率、能源依赖和经济发展基础。

为此，本文选取 CO₂ 总量、人均 CO₂、碳排放强度、煤炭相关排放占比、第二产业占比、人均 GDP 和城镇化率作为核心指标。先通过省级空间热力图直观呈现排放的地理集聚格局，再用变异系数、基尼系数和泰尔指数刻画省际差异显著性；随后构建熵权 TOPSIS 模型，利用客观权重得到各省低碳发展综合得分；最后采用 K-means 聚类对省份进行分类，使“空间分布-差异测度-综合评价-分类分级”形成完整分析链条。

2.2 问题二的分析

问题二的核心是从多个候选因素中识别碳排放的主要驱动因素，并建立能够解释省际排放差异的定量模型。碳排放通常受到人口规模、经济活动水平、产业结构、能源消费结构和城镇化进程共同影响，这些因素之间并非相互独立。例如，人口规模扩大会推高能源需求，人均 GDP 提高会改变消费和生产活动强度，第二产业占比较高通常意味着更强的能源消耗，煤炭占比较高则会直接提高单位能源消费对应的碳排放。因此，模型不仅要估计各因素的方向和强弱，还要检验变量间相关性对结论稳定性的影响。

为此，本文采用 STIRPAT 模型作为问题二的基本框架，将碳排放总量对人口、人均 GDP、煤炭相关排放占比、第二产业占比和城镇化率的弹性关系转化为对数线性回归模型。模型参数采用 OLS 估计，并通过回归系数、标准误、t 统计量和 p 值判断变量影响方向及显著性；同时引入 VIF 检验多重共线性，避免解释变量高度相关造成误判。考虑到省级横截面样本量有限，进一步采用岭回归和留一交叉验证作为稳健性检验，从而判断驱动因素识别结论是否可靠。

2.3 问题三的分析

问题三要求在未来较长时期内预测全国碳排放总量和碳排放强度，并进一步判断碳达峰时间、峰值水平和减排潜力。该问题不同于前两问的横截面分析，它需要把驱动因素的影响关系转化为年度变化路径。由于全国年度样本较短，若直接建立复杂多变量时间序列模型，容易出现参数不稳定和外推误差较大的问题。同时，未来人口、经济增长、能源结构和产业结构本身具有不确定性，单一路径预测难以反映不同政策强度下的排放差异。

为此，本文以 2024 年全国实际排放量作为预测基准，将问题二得到的 STIRPAT 弹性系数转化为年度相对变化递推模型，使人口增长率、人均 GDP 增长率、煤炭占比变化率、第二产业占比变化率和城镇化率变化率共同决定下一期排放水平。在情景设置上，分别构建基准、低碳和强化低碳三种发展路径，以体现不同能源替代速度和产业转型强度。最后根据各情景预测序列识别峰值年份和峰值排放量，并计算 2045 年相对峰值的减排比例；同时进行单因素和组合扰动灵敏度分析，检验关键参数变化对预测结论的影响。

2.4 问题四的分析

问题四要求在前三问定量分析基础上提出政策建议，因此它不是简单的文字总结，而是需要把空间差异、分类分级、驱动因素和未来情景预测连接起来。不同类型省份的排放压力来源并不相同：资源依赖型地区主要受煤炭消费和能源结构制约，工业制造型地区更多受高耗能行业和产业结构影响，经济发达效率型地区则具备技术、资金和治理优势。若采用统一政策口径，容易造成高排放省份减排压力不足或低排放省份承担过高转型成本。

为此，本文将问题一得到的省份类型作为政策分区依据，将问题二识别出的煤炭相关排放占比、人口规模等关键驱动因素作为政策抓手，并结合问题三中三种情景下的达峰时间和减排潜力判断政策强度。政策建议按照“资源依赖高排放型、工业制造高排放

型、中等转型压力型、经济发达效率型”分别提出，重点围绕控煤降碳、重点行业节能改造、防止高耗能产业转移、绿色金融和碳市场机制展开，从而形成具有区域针对性和可操作性的减排路径。

三、模型假设

1. 附件中碳排放数据真实可靠，各省统计口径在同一年份内具有可比性。
2. 2022 年省级横截面数据能够代表当前我国省际碳排放差异和主要结构特征。
3. 人口规模、经济发展水平、能源结构、产业结构和城镇化率是影响碳排放的重要因素，且可通过 STIRPAT 模型进行定量刻画。
4. 全国中长期碳排放趋势主要由人口、人均经济水平、煤炭占比、产业结构和城镇化率共同决定。
5. 情景预测中，各驱动变量变化率在 2026–2045 年间平滑变化，不考虑突发重大外部冲击。
6. 附件 1 中 2025 年全国碳排放数据截至 2025 年 9 月 30 日，不能直接作为完整年度数据与 2019–2024 年比较，预测基准采用 2024 年全国排放量。

四、符号说明

表 1 主要符号说明

符号	说明	单位
C_i	第 i 个省份碳排放总量	Mt
P_i	第 i 个省份人口规模	万人
GDP_i	第 i 个省份地区生产总值	亿元
PC_i	第 i 个省份人均碳排放	吨/人
CI_i	第 i 个省份碳排放强度	吨/万元 GDP
$Coal_i$	第 i 个省份煤炭相关排放占比	%
IS_i	第 i 个省份第二产业占比	%
A_i	第 i 个省份人均 GDP	万元/人
U_i	第 i 个省份城镇化率	%
z_{ij}	第 i 个省份第 j 个指标标准化值	—
w_j	第 j 个评价指标权重	—
S_i	第 i 个省份 TOPSIS 综合贴近度	—
G	基尼系数	—
T	泰尔指数	—
CV	变异系数	—
g_P	人口增长率	%
g_A	人均 GDP 增长率	%
$\Delta Coal$	煤炭相关排放占比变化量	百分点
ΔIS	第二产业占比变化量	百分点
ΔU	城镇化率变化量	百分点

五、问题一的模型建立与求解

5.1 指标选取与基础变量构造

问题一要求同时回答“空间差异是否显著”和“省份应如何分类分级”。为避免只用排放总量造成片面判断，本文从排放规模、人口压力、经济效率、能源结构、产业结构和发展水平六个方面构建指标体系。具体变量包括碳排放总量 (C_i)、人口规模 (P_i)、地区生产总值 (GDP_i)、人均碳排放 (PC_i)、碳排放强度 (CI_i)、煤炭相关排放占比 ($Coal_i$)、第二产业占比 (IS_i)、人均 GDP (A_i) 和城镇化率 (U_i)。其中， C_i 反映排放规模， PC_i 反映居民平均排放压力， CI_i 反映单位经济产出的排放效率， $Coal_i$ 和 IS_i 分别反映能源结构与产业结构， A_i 和 U_i 反映发展水平和治理基础。

在原始变量基础上，首先构造人均碳排放、碳排放强度和人均 GDP 三个派生指标：

$$\begin{cases} PC_i = \frac{C_i}{P_i}, \\ CI_i = \frac{C_i}{GDP_i}, \\ A_i = \frac{GDP_i}{P_i}. \end{cases}$$

第一式表示第 i 省的居民平均排放水平；第二式表示单位 GDP 对应的碳排放量，数值越低说明经济产出效率越高；第三式表示第 i 省的经济发展水平。这样处理后，可以将人口规模和经济规模差异从总量指标中分离出来，使省际比较更公平。

5.2 空间差异测度模型

为判断省际碳排放差异是否显著，本文对核心指标 x_i 分别计算变异系数、基尼系数和泰尔指数。变异系数用于衡量相对离散程度，基尼系数用于衡量省际分布不均衡程度，泰尔指数可进一步刻画差异强度。设 x_i 表示第 i 省某一碳排放指标， μ_x 和 σ_x 分别为该指标的均值和标准差，则变异系数为

$$CV = \frac{\sigma_x}{\mu_x}.$$

当 CV 较大时，说明各省之间相对差异明显。

将 x_i 按从小到大排序后，基尼系数定义为

$$G = \frac{2 \sum_{i=1}^n i x_i}{n \sum_{i=1}^n x_i} - \frac{n+1}{n}.$$

该式通过排序后各省指标值的累积分布刻画不均衡程度， G 越大，说明指标越集中于少数省份。

泰尔指数定义为

$$T = \frac{1}{n} \sum_{i=1}^n \frac{x_i}{\bar{x}} \ln \left(\frac{x_i}{\bar{x}} \right).$$

其中， $\bar{x}$ 为指标均值。当某些省份指标值显著高于均值时，泰尔指数会增大，因此可作为差异显著性的补充度量。

5.3 熵权 TOPSIS 综合评价模型

空间差异指标只能说明“差异有多大”，不能直接判断各省低碳发展水平的综合优劣。因此，本文进一步构建熵权 TOPSIS 模型 [1]。设第 i 省第 j 个指标的原始值为 x_{ij} ，评价指标集合为

$$X_i = (C_i, PC_i, CI_i, Coal_i, IS_i, A_i, U_i).$$

其中, C_i 、 PC_i 、 CI_i 、 $Coal_i$ 和 IS_i 越小越有利于低碳发展, 作为负向指标; A_i 和 U_i 体现经济基础和城市治理能力, 作为正向指标。

首先对指标进行无量纲化处理。正向指标和负向指标分别采用如下标准化公式:

$$z_{ij} = \begin{cases} \frac{x_{ij} - \min_i x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}, & \text{正向指标,} \\ \frac{\max_i x_{ij} - x_{ij}}{\max_i x_{ij} - \min_i x_{ij}}, & \text{负向指标.} \end{cases}$$

该式把不同量纲的指标统一映射到 $[0, 1]$ 区间, 并保证数值越大代表低碳表现越好。

然后计算熵权。令 $p_{ij} = z_{ij} / \sum_{i=1}^n z_{ij}$, 则第 j 个指标的熵值和权重分别为

$$e_j = -\frac{1}{\ln n} \sum_{i=1}^n p_{ij} \ln p_{ij}, \quad w_j = \frac{1 - e_j}{\sum_{j=1}^m (1 - e_j)}.$$

熵值 e_j 越小, 说明该指标在省际之间差异越明显、提供的信息量越大, 因此对应权重 w_j 越高。

最后计算各省到理想解和负理想解的距离。设 $z_j^+ = \max_i z_{ij}$ 为理想解, $z_j^- = \min_i z_{ij}$ 为负理想解, 则

$$D_i^+ = \sqrt{\sum_{j=1}^m w_j (z_{ij} - z_j^+)^2}, \quad D_i^- = \sqrt{\sum_{j=1}^m w_j (z_{ij} - z_j^-)^2}.$$

据此得到 TOPSIS 贴近度

$$S_i = \frac{D_i^-}{D_i^+ + D_i^-}.$$

S_i 越接近 1, 表示第 i 省越接近低碳理想状态; S_i 越接近 0, 说明其排放压力或结构压力越大。

5.4 K-means 分类模型

在得到综合评价结果后, 还需要将省份划分为若干具有政策含义的类型。本文采用 K-means 聚类算法 [2], 仍以 $X_i = (C_i, PC_i, CI_i, Coal_i, IS_i, A_i, U_i)$ 作为聚类变量。该变量组同时覆盖排放规模、排放效率、能源结构、产业结构和发展基础, 能够反映不同省份减排压力来源的差异。

设 G_k 为第 k 类省份集合, μ_k 为第 k 类中心, 聚类目标为最小化类内平方误差和:

$$J = \sum_{k=1}^K \sum_{i \in G_k} \|X_i - \mu_k\|^2.$$

当 J 越小时, 同一类型内部省份越相似。为辅助判断聚类数 K , 本文计算轮廓系数

$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}.$$

其中, $a(i)$ 为省份 i 与同类省份的平均距离, $b(i)$ 为省份 i 与最近其他类省份的平均距离。 $s(i)$ 越接近 1, 说明该省份分类越合理。

5.5 求解结果与分析

基于 2022 年 30 省数据，首先计算空间差异指标，结果如表2所示。

表 2 省际碳排放空间差异指标

指标	均值	变异系数	基尼系数	泰尔指数	最小值	最大值
CO ₂ 总量/Mt	380.2822	0.6458	0.3549	0.2041	44.6919	920.9053
人均 CO ₂ /吨每人	9.8017	0.7957	0.3513	0.2312	3.2763	36.6002
碳排放强度/吨每万元 GDP	1.3011	0.7958	0.3883	0.2555	0.1720	4.8805
煤炭相关排放占比	0.7113	0.2510	0.1263	0.0447	0.0212	0.9399

结合图2可以看出，2022 年我国 30 省碳排放呈现明显地理集聚特征。高排放区域集中在华北-环渤海及内蒙古能源基地一带，东部沿海制造业省份也形成高排放带，中西部和西南多数省份排放总量相对较低。因此，省际碳排放空间格局可概括为“北方资源能源高排放带、东部沿海制造业高排放带、中西南相对低排放区、局部资源型高值点”。

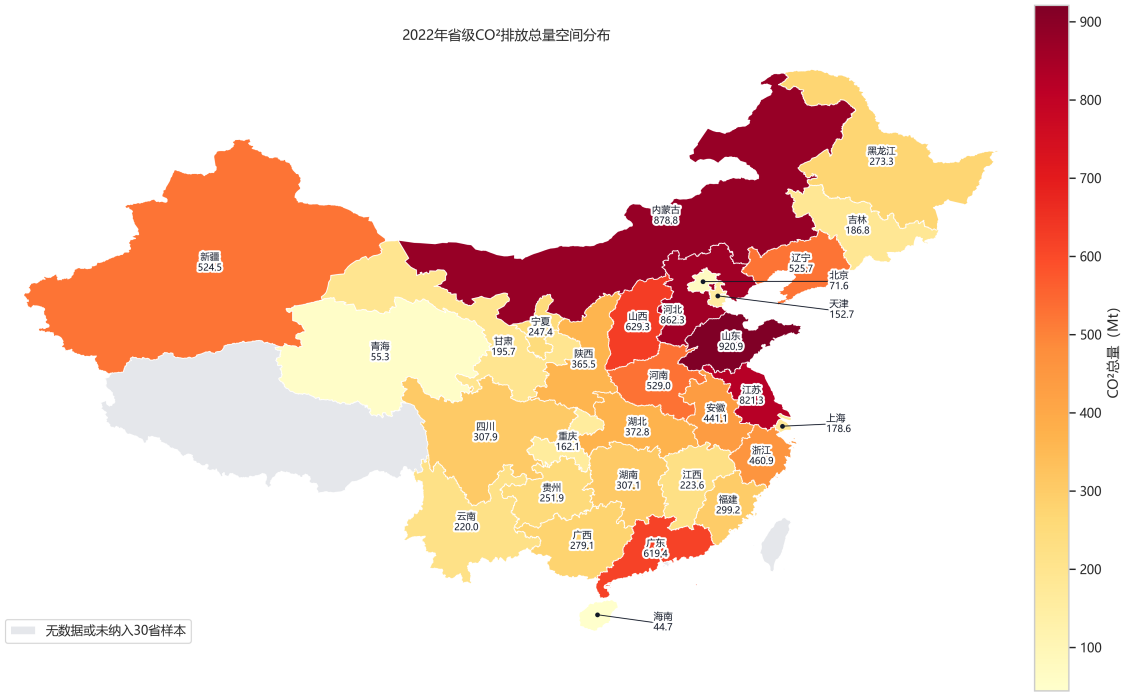


图 2 2022 年省级 CO₂ 排放总量空间分布热力图

差异显著性方面，2022 年 30 省 CO₂ 总量均值为 380.2822 Mt，但最大值山东为 920.9053 Mt，最小值海南为 44.6919 Mt，二者相差约 20.61 倍；排放总量前 8 位省份合计占 30 省总排放的 50.72%，说明排放明显集中在少数重点省份。碳排放强度基尼系数为 0.3883，高于总量基尼系数，说明省际排放效率差异更突出。进一步观察图3可知，人均 CO₂ 和碳排放强度并非简单随总量同步变化。资源依赖型省份通常处于图中右上方，表现为人均排放和排放强度同时偏高；部分沿海制造业省份虽然排放总量较大，但人均排放或碳排放强度未必同步最高，说明其减排重点更偏向产业结构和能源效率优化。

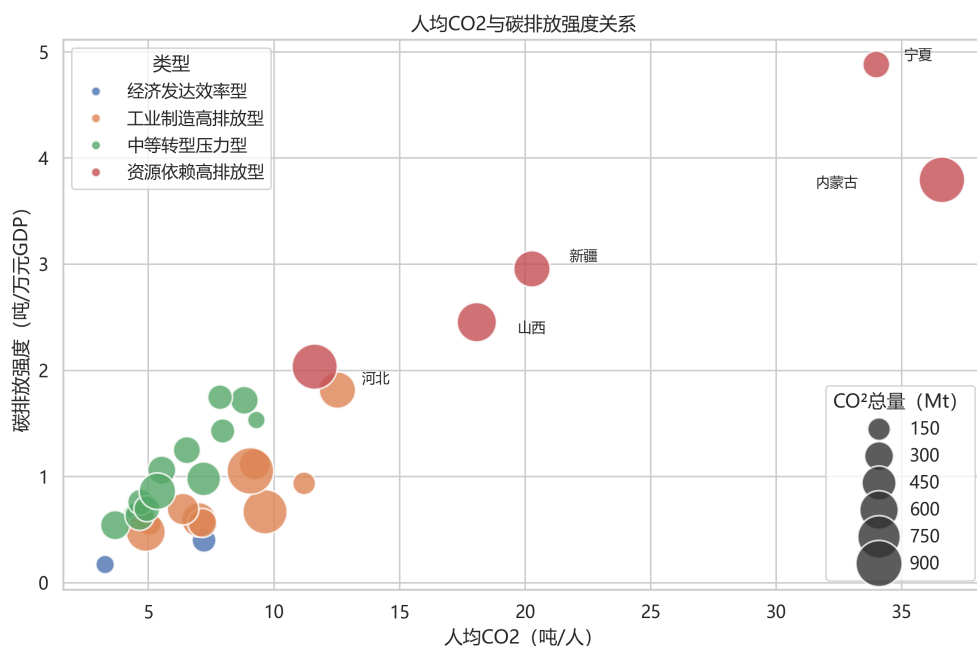


图 3 人均 CO₂ 与碳排放强度散点图

熵权 TOPSIS 评价结果显示，指标权重前三位为人均 GDP、煤炭相关排放占比和城镇化率，权重分别为 0.2728、0.2215 和 0.1717。由图4可见，TOPSIS 得分排名靠前的省份为北京、上海、天津、海南、浙江、福建、重庆、广东，这些省份通常具有较高经济发展水平、较低碳排放强度或较好的能源结构；排名靠后的省份为山西、内蒙古、新疆、河北、宁夏、河南、山东、甘肃，主要受煤炭依赖、排放强度偏高或总量压力较大影响。该结果说明，低碳综合水平并不只由排放总量决定，而是由经济效率、能源结构和发展阶段共同决定。

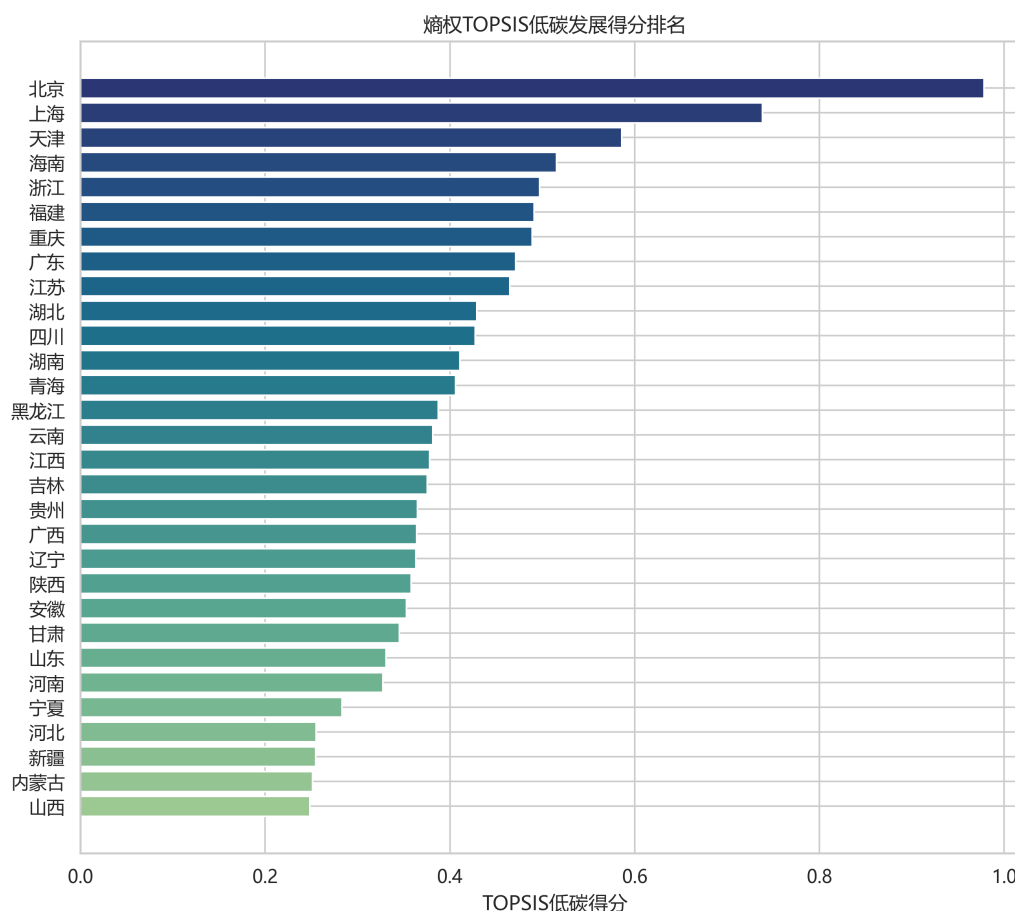


图 4 TOPSIS 低碳综合得分排名图

在综合评价基础上，进一步进行 K-means 聚类。聚类模型中 K=3 时轮廓系数最高，但 K=4 能够更清晰地区分资源依赖、工业制造、中等转型和经济发达效率型省份，政策解释性更强。因此本文采用 K=4 作为最终分类，结果如表3所示。

表 3 K-means 聚类分类结果

类型	省份	结果分析
资源依赖高排放型	河北、山西、内蒙古、宁夏、新疆	煤炭占比和排放强度高，减排重点是控煤和资源型产业转型
工业制造高排放型	天津、辽宁、江苏、浙江、福建、山东、湖北、广东、重庆、陕西	排放总量较高，减排重点是重点行业节能改造
中等转型压力型	吉林、黑龙江、安徽、江西、河南、湖南、广西、海南、四川、贵州、云南、甘肃、青海	总量和强度居中，需防止高耗能产业转移
经济发达效率型	北京、上海	排放强度低，适合发挥技术、金融和治理优势

为进一步解释各类型的内部特征，本文绘制四类省份的标准化均值画像，如图5所示。资源依赖高排放型在煤炭占比、人均 CO₂ 和碳排放强度上明显偏高，说明其减排约束主要来自能源结构；工业制造高排放型在总量和第二产业占比上较突出，说明其重

点在工业部门节能降碳；中等转型压力型各指标整体居中，应防止高耗能产业转移造成排放反弹；经济发达效率型在人均 GDP 和城镇化率上优势明显，同时排放强度较低，适合作为绿色技术、绿色金融和碳市场机制的示范区域。

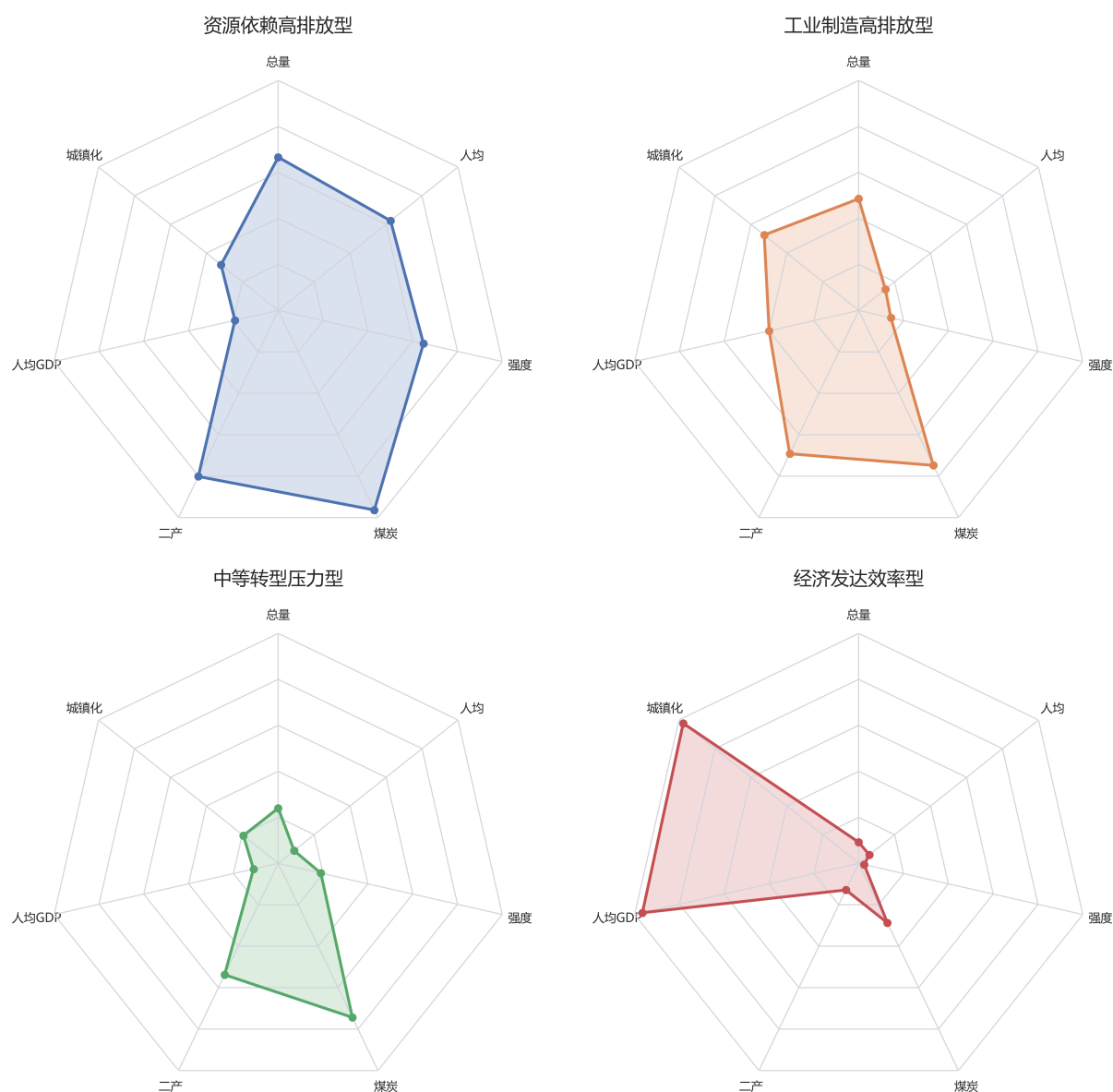


图 5 K-means 四类省份标准化均值画像雷达图

六、问题二的模型建立与求解

6.1 驱动因素选择

问题二要求识别影响碳排放的核心驱动因素，并建立可检验的定量模型。结合 STIRPAT 理论和省级数据可得性，本文以碳排放总量 (C_i) 为被解释变量，选取人口规模 (P_i)、人均 GDP (A_i)、煤炭相关排放占比 ($Coal_i$)、第二产业占比 (IS_i) 和城镇化率 (U_i) 作为解释变量。人口规模 (P_i) 代表社会经济活动总量，人均 GDP (A_i) 代表富裕程度和消费生产强度，煤炭相关排放占比 ($Coal_i$) 代表能源结构，第二产业占比 (IS_i) 代表产业结构，城镇化率 (U_i) 代表发展阶段和城市能源需求。

6.2 STIRPAT 模型构建

STIRPAT 模型由 IPAT 恒等式扩展而来 [3]，本文将其写为

$$I = aP^b A^c T^d e.$$

式中， I 表示环境影响，本文对应碳排放总量； P 表示人口规模； A 表示富裕程度； T 表示技术和结构因素； a 为常数项， b, c, d 为弹性系数， e 为随机扰动项。由于能源结构、产业结构和城镇化率难以合并为单一技术变量，本文将 T 拆分为 $Coal_i$ 、 IS_i 和 U_i 三个可观测变量。

对模型两边取对数，并结合省级横截面数据，得到本文的扩展 STIRPAT 模型：

$$\ln C_i = \alpha + \beta_1 \ln P_i + \beta_2 \ln A_i + \beta_3 Coal_i + \beta_4 IS_i + \beta_5 U_i + \varepsilon_i.$$

其中， β_1 表示人口规模对碳排放的弹性， β_2 表示人均 GDP 对碳排放的弹性； β_3 、 β_4 和 β_5 分别表示煤炭占比、第二产业占比和城镇化率变化对碳排放对数值的边际影响。若某一系数为正，说明该因素增加会推动碳排放上升；若为负，则说明该因素具有抑制排放的作用。

6.3 参数估计与模型检验

令 $y_i = \ln C_i$ ， $\hat{y}_i$ 为模型拟合值，OLS 估计通过最小化残差平方和得到参数：

$$\hat{\beta}_{OLS} = \arg \min_{\beta} \sum_{i=1}^n (y_i - \hat{y}_i)^2.$$

该方法使预测值与真实值之间的总体误差最小，用于获得 STIRPAT 主模型参数。

考虑到解释变量之间可能存在相关性，本文进一步计算方差膨胀因子：

$$VIF_j = \frac{1}{1 - R_j^2}.$$

其中， R_j^2 为第 j 个解释变量对其余解释变量回归得到的决定系数。 VIF_j 越大，说明该变量与其他解释变量相关性越强，多重共线性风险越高。

为检验模型稳健性，本文采用岭回归作为补充。岭回归目标函数为

$$\hat{\beta}_{ridge} = \arg \min_{\beta} \left\{ \sum_{i=1}^n (y_i - \hat{y}_i)^2 + \lambda \sum_{j=1}^p \beta_j^2 \right\}.$$

其中， λ 为惩罚系数。该式在残差平方和基础上加入系数平方惩罚，可以降低多重共线性和小样本波动对参数估计的影响。

6.4 求解结果与分析

估计结果见表4。模型 R^2 为 0.8762，调整 R^2 为 0.8504，RMSE_log 为 0.2672，F 检验 p 值为 3.919×10^{-10} ，说明模型对省际碳排放差异具有较强解释力。

表 4 STIRPAT 模型 OLS 估计结果

变量	系数	标准误	t 值	p 值
$\ln P$	0.6447	0.0807	7.9856	0.0000
$\ln A$	0.4502	0.3860	1.1663	0.2549
煤炭相关排放占比	2.8275	0.5798	4.8770	0.0001
第二产业占比	0.5080	1.1516	0.4412	0.6630
城镇化率	0.8029	1.3461	0.5965	0.5565

从参数估计结果看，人口规模和煤炭相关排放占比对碳排放具有显著正向影响。其中， $\ln P$ 系数为 0.6447，煤炭相关排放占比系数为 2.8275，且二者 p 值较小，是当前样本中最稳定的排放驱动因素。VIF 检验显示所有解释变量 VIF 均小于 10，不存在严重多重共线性。岭回归留一交叉验证得到 $LOOCV_R^2$ 为 0.7851，说明模型具有一定预测稳健性。

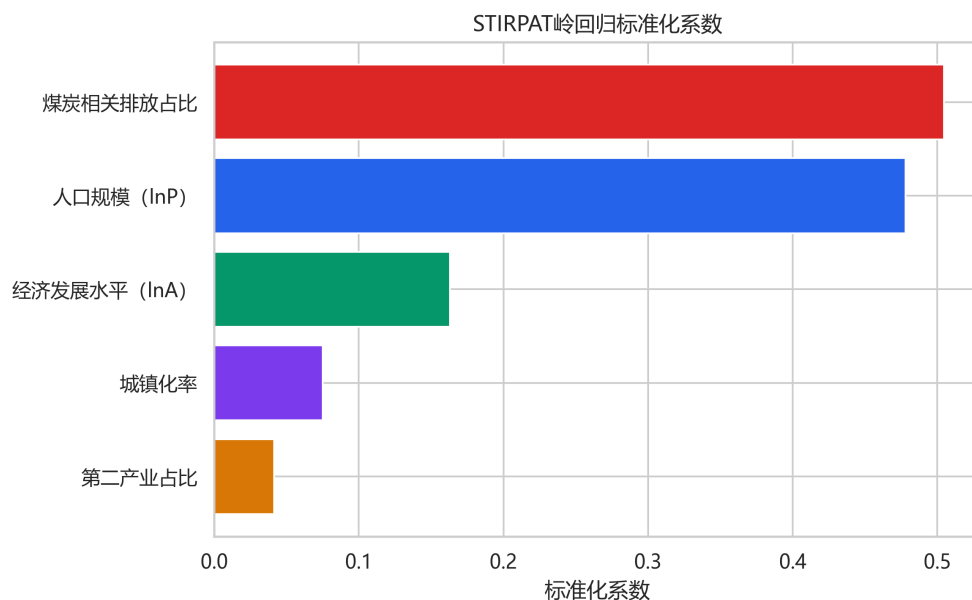


图 6 STIRPAT 岭回归标准化系数

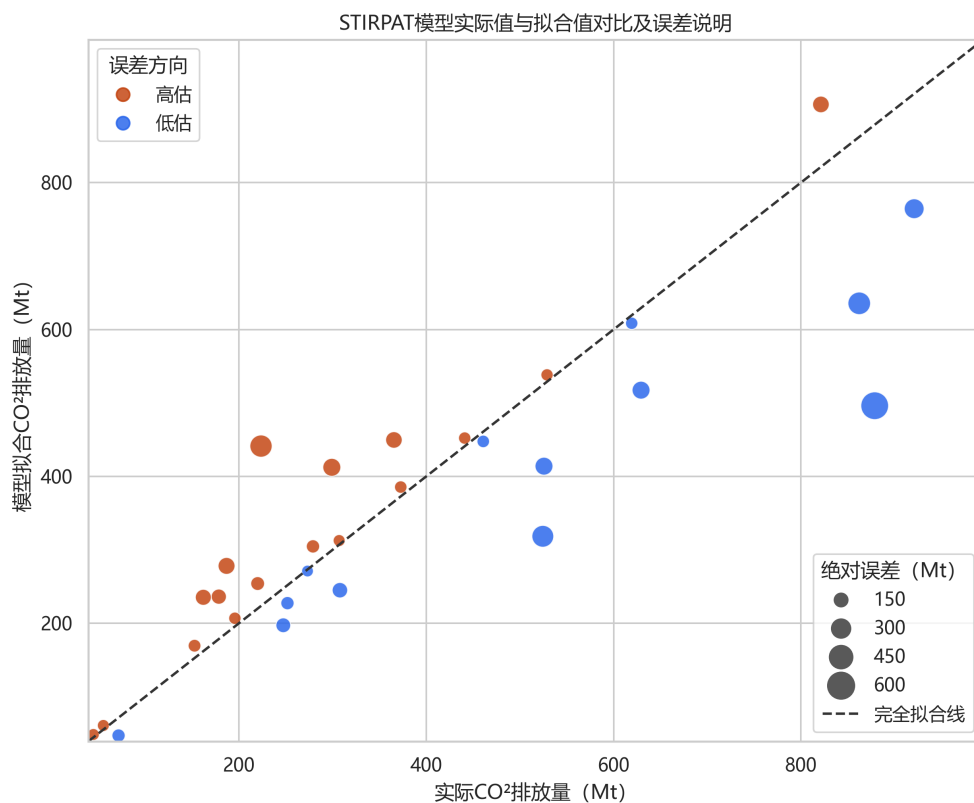


图 7 OLS 拟合值与真实值对比

七、问题三的模型建立与求解

7.1 预测变量与建模思路

问题三要求预测 2026–2045 年全国碳排放总量与强度，并判断达峰时间、峰值水平和减排潜力。与问题二的省级横截面模型不同，问题三需要把驱动因素关系转化为年度递推过程。本文使用全国碳排放总量 (C_t)、人口规模 (P_t)、人均 GDP (A_t)、煤炭相关排放占比 ($Coal_t$)、第二产业占比 (IS_t) 和城镇化率 (U_t) 作为预测变量。其中， P_t 和 A_t 反映人口与经济增长，通常会推高排放； $Coal_t$ 和 IS_t 反映能源结构和产业结构，是低碳转型的主要调节变量； U_t 反映城镇化发展阶段。

由于全国年度样本较短，直接建立多变量时间序列回归会使参数不稳定。因此，本文沿用问题二得到的 STIRPAT 系数，将其转化为相对变化递推模型。该做法的含义是：未来某一年排放变化不是单独由时间趋势决定，而是由人口、经济、能源结构、产业结构和城镇化率的年度变化共同决定。

7.2 STIRPAT 递推预测模型

设 $\Delta \ln C_t = \ln C_t - \ln C_{t-1}$ ， $\Delta Coal_t = Coal_t - Coal_{t-1}$ ，其余变量类似，则年度排放变化由下式给出：

$$\Delta \ln C_t = \beta_1 \Delta \ln P_t + \beta_2 \Delta \ln A_t + \beta_3 \Delta Coal_t + \beta_4 \Delta IS_t + \beta_5 \Delta U_t.$$

该式把问题二估计得到的驱动因素系数用于年度变化分析：人口和人均 GDP 采用对数变化率，煤炭占比、第二产业占比和城镇化率采用比例变化量。

由 $\Delta \ln C_t$ 可以递推出下一年排放总量：

$$\begin{cases} C_t = C_{t-1} \exp(\Delta \ln C_t), \\ GDP_t = P_t A_t, \\ CI_t = \frac{C_t}{GDP_t}. \end{cases}$$

第一式表示以 $t-1$ 年排放总量为基础，根据驱动因素变化得到 t 年排放；第二式表示全国 GDP 由人口和人均 GDP 相乘得到；第三式表示碳排放强度，即单位 GDP 对应的碳排放量。

达峰年份和减排潜力定义为

$$Y_{peak} = \arg \max_t C_t, \quad R_{2045} = 1 - \frac{C_{2045}}{\max_t C_t}.$$

其中， Y_{peak} 表示预测期内排放总量达到最大值的年份； R_{2045} 表示 2045 年相对峰值排放的下降比例，数值越大说明减排潜力越充分。

7.3 情景参数设置

全国年度碳排放总量仅使用附件 1 中 $Sector = Total$ 统计。2019–2024 年全国碳排放整体处于高位波动状态，2024 年为 11901.44 Mt。2025 年只有 273 天记录，不能作为完整年度值进行同比比较。因此，本文以 2024 年全国排放量 11901.44 Mt 作为情景预测基准。三种情景参数设置见表 5。

基准情景假设现有转型速度延续，经济保持中速增长，煤炭占比和第二产业占比逐步下降；低碳情景假设能源替代和产业优化速度进一步提高；强化低碳情景假设政策约束、技术进步和非化石能源替代力度显著增强。三种情景均保持人口增长率逐步下降的趋势，以反映我国人口变化现实。

表 5 三情景参数设置

情景	人口增长率	人均 GDP 增长率	煤炭占比年变化	第二产业占比年变化	城镇化率年变化
基准情景	-0.10%→-0.20%	4.50%→2.60%	-0.40→-1.00 百分点	-0.05→-0.20 百分点	0.35→0.15 百分点
低碳情景	-0.10%→-0.20%	4.30%→2.40%	-0.60→-1.40 百分点	-0.10→-0.30 百分点	0.30→0.10 百分点
强化低碳情景	-0.10%→-0.25%	4.10%→2.20%	-0.80→-1.80 百分点	-0.20→-0.50 百分点	0.25→0.05 百分点

7.4 预测结果与检验

预测结果见表6和图8。基准情景下，碳排放在 2032 年达到约 12435 Mt 峰值，随后缓慢下降，2045 年较峰值下降 11.4%。低碳情景下，排放约在 2026 年达到峰值 11959 Mt，2045 年较峰值下降 25.6%。强化低碳情景下降幅最大，2045 年降至 7114 Mt，较峰值下降 40.2%。

表 6 三情景达峰与减排潜力

情景	达峰年份	峰值/Mt	2030 年/Mt	2035 年/Mt	2045 年/Mt	2045 较峰值下降
低碳情景	2026	11959	11829	11229	8903	25.6%
基准情景	2032	12435	12404	12351	11014	11.4%
强化低碳情景	2024	11901	11258	10165	7114	40.2%

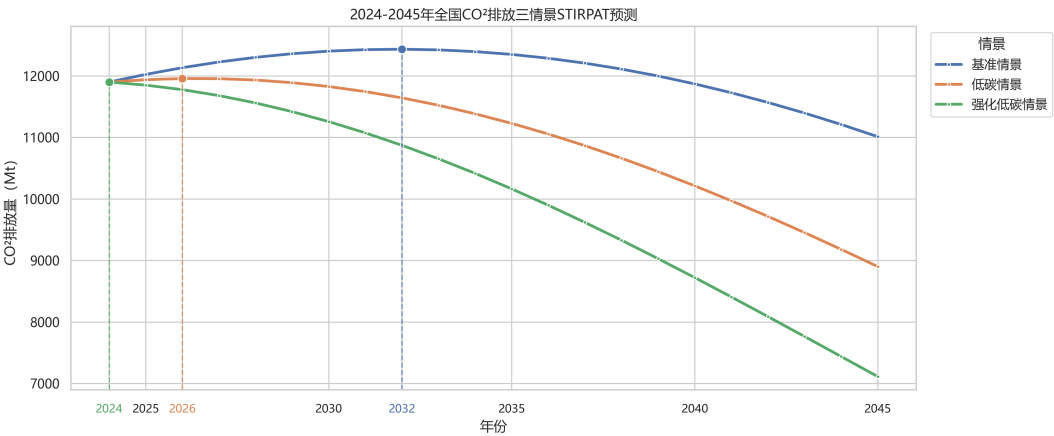


图 8 2024–2045 年三情景碳排放趋势

7.5 灵敏度分析

由于情景预测结果依赖未来人口、经济、能源结构和产业结构变化假设，且 STIRPAT 系数本身来自有限省级样本估计，因此有必要检验关键参数扰动对预测结论的影响。本文从两个层面进行灵敏度分析：第一，对情景参数进行单因素扰动，观察 2045 年排放量和达峰年份是否明显变化；第二，对 STIRPAT 模型系数进行扰动，检验驱动因素弹性估计变化是否会改变主要预测判断。

灵敏度分析表明，2045 年排放量对煤炭占比下降速度和人均 GDP 增长率最敏感。当煤炭占比下降速度减弱 20% 时，2045 年排放量增加约 954.7 Mt，达峰年份推迟至 2035 年；当煤炭占比下降速度增强 20% 时，2045 年排放量减少约 878.6 Mt，达峰年份提前至 2030 年。主要结论在合理扰动范围内未发生根本改变，说明模型具有一定稳健性。

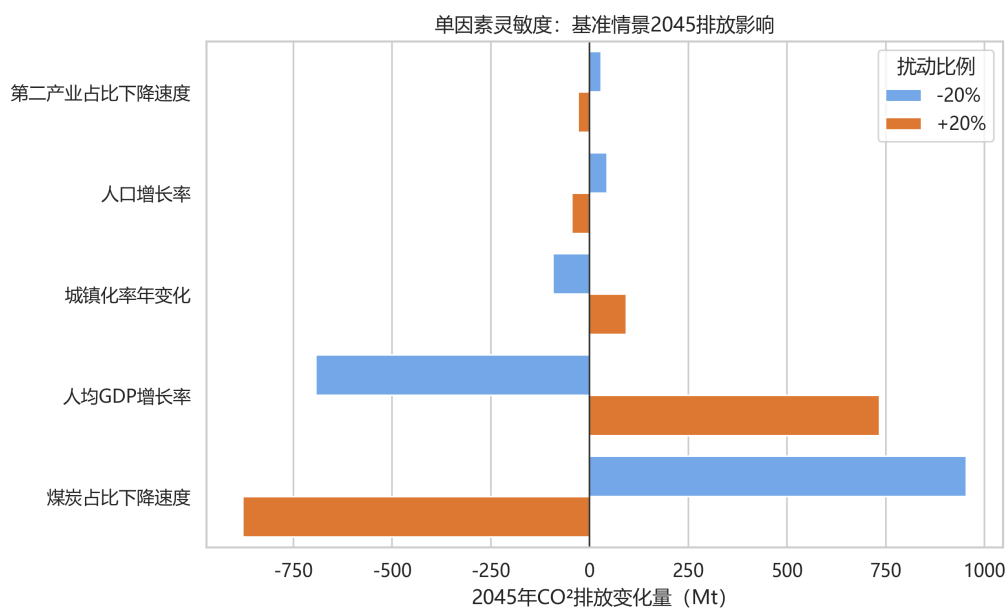


图 9 单因素灵敏度对 2045 年排放的影响

进一步地，图10展示了 STIRPAT 系数在正负扰动下对 2045 年排放量的影响。可以看出，煤炭相关排放占比系数和人均 GDP 系数的扰动对预测结果影响较大，说明能源结构调整速度和经济增长方式是长期排放路径中最关键的不确定来源；人口规模、第二产业占比和城镇化率系数扰动的影响相对较小，表明这些因素虽然参与排放递推，但在本文设定的变化范围内不会单独改变总体达峰判断。因此，问题三的核心结论具有一定稳定性，即强化低碳情景能够显著提前达峰并提高 2045 年减排幅度，而基准情景下达峰时间偏晚、减排潜力不足。

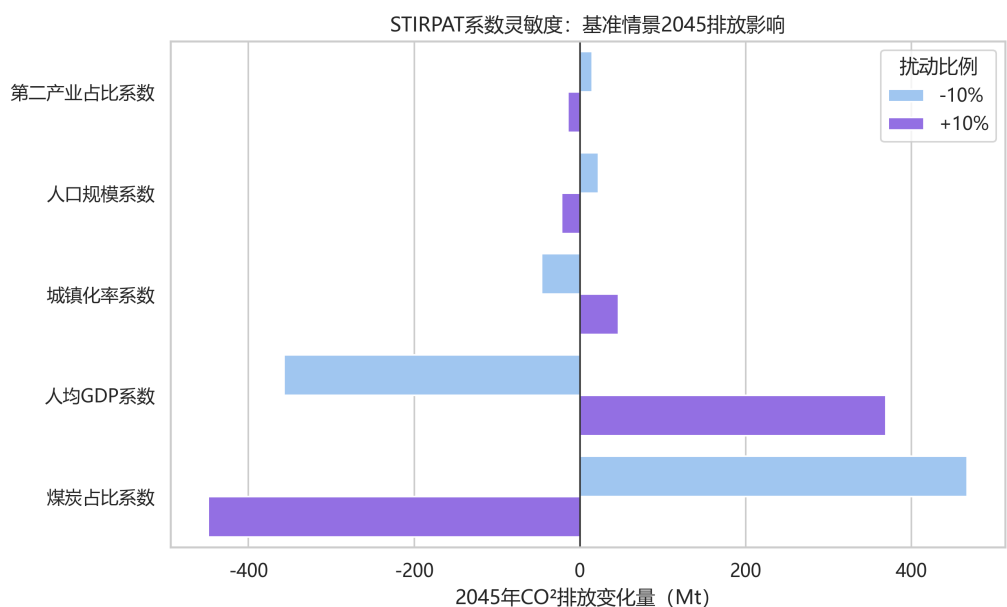


图 10 STIRPAT 系数灵敏度对 2045 年排放的影响

“双碳” 目标实现路径与区域减排建议

依据省际空间差异、STIRPAT驱动因素识别和2026—2045年情景预测结果，形成面向能源、产业、区域和机制的综合政策方案。

2032年

基准情景达峰时间偏晚，峰值约12435 Mt。

40.2%

强化低碳情景下2045年较峰值下降幅度。

煤炭占比

模型识别出的关键结构性驱动因素。

综合建议

结合问题一至三的研究结论，我国推进“双碳”目标应坚持“控总量、调结构、提效率、分区域、强机制”的路径。首先，优化能源结构是达峰和深度减排的核心，应持续压降煤炭相关排放占比，推动煤电灵活性改造和清洁高效利用，同时扩大风电、光伏、水电、核电及储能等非化石能源供给。其次，推动产业升级应聚焦钢铁、建材、电力、石化和交通等重点行业，建立单位产品碳强度约束，推广余热利用、电气化替代、绿色供应链和循环经济。再次，区域减排应分类实施：河北、山西、内蒙古、宁夏、新疆等资源依赖高排放型省份重点控煤和转型；江苏、山东、广东、辽宁等工业制造高排放型省份重点推进重点行业节能改造；中等转型压力型省份严控高耗能产业转移；北京、上海发挥绿色金融、碳交易和数字化治理示范作用。最后，应强化低碳技术创新和政策机制衔接，完善碳市场、绿色金融、能耗和碳排放双控考核，形成可量化、可追踪、可持续的减排体系。

能源结构

以煤炭替代和非化石能源扩张为主线，优先降低高煤依赖省份的边际排放。

产业升级

围绕高耗能行业开展节能改造，以强度约束倒逼生产方式绿色转型。

区域协同

按省份类型制定差异化政策，避免“一刀切”和高耗能产业简单转移。

机制保障

以碳市场、绿色金融和数字化碳管理提高政策执行的可考核性。

八、模型的评价与改进

8.1 模型的优点

1. 模型体系完整。本文从空间差异、综合评价、分类分级、驱动因素识别、趋势预测和政策建议六个层面展开，能够完整回应题目要求。
2. 解释性较强。熵权 TOPSIS、K-means 聚类 and STIRPAT 模型均具有较清晰的经济含义，便于解释不同省份的低碳发展水平和排放驱动机制。
3. 结果展示直观。空间差异指标、TOPSIS 排名、聚类类型、回归系数和情景预测结果都通过表格和图形呈现，便于比较和分析。
4. 情景预测逻辑稳定。STIRPAT 递推模型将问题二估计得到的驱动因素系数直接用于问题三预测，使驱动因素识别与中长期趋势研判形成衔接。

8.2 模型的缺点

1. 省级建模主要基于 2022 年横截面数据，尚不能充分刻画各省碳排放随时间变化的动态机制。若能补充多年省级面板数据，可进一步建立双向固定效应 STIRPAT 模型。
2. STIRPAT 模型采用线性对数形式，难以反映复杂非线性关系和变量之间的交互效应。后续可尝试空间计量模型或非线性机器学习模型进行补充检验。
3. 情景预测依赖参数设定，不同参数组合会影响达峰年份和减排潜力判断。本文已通过灵敏度分析检验主要结论稳健性，但未来仍可进一步扩大参数组合范围。
4. 当前模型主要分析省际差异，没有进一步考虑相邻地区之间的空间溢出效应。后续可引入空间自相关检验和空间杜宾模型，对区域联动减排进行深入研究。

参考文献

- [1] Deng H, Yeh C H, Willis R J, Inter-company comparison using modified TOPSIS with objective weights, Computers & Operations Research, 27(10): 963-973, 2000。
- [2] MacQueen J, Some methods for classification and analysis of multivariate observations, Proceedings of the Fifth Berkeley Symposium on Mathematical Statistics and Probability, 1: 281-297, 1967。
- [3] York R, Rosa E A, Dietz T, STIRPAT, IPAT and ImPACT: analytic tools for unpacking the driving forces of environmental impacts, Ecological Economics, 46(3): 351-365, 2003。

附录

本文计算结果由 Python 程序统一生成。附录列出主要求解脚本，但已删去生成 README、项目汇总说明、gitignore 等项目文档的工程性代码，仅保留与数据读取、模型计算、结果检验和图表输出相关的程序。

附录 A 总入口脚本 run_all.py

```
1 from __future__ import annotations
2
3 """
4 总入口脚本
5
6 作用：
7 - 依次调用问题一、问题二、问题三、问题四的独立脚本。
8 - 生成分类目录下的结果表、图片、政策建议和汇总报告。
9 - 执行必要的输出完整性检查。
10
11 运行方式：
12 .\\.venv\\Scripts\\python.exe .\\3. 代码\\run_all.py
13 """
14
15 import sys
16 from pathlib import Path
17
18 CODE_ROOT = Path(__file__).resolve().parent
19 ROOT = CODE_ROOT.parent
20 sys.pycache_prefix = str(ROOT / ".pycache")
21 sys.path.append(str(CODE_ROOT))
22
23 import numpy as np
24 import pandas as pd
25
26 from common import (
27     FIG_DIR,
28     PROBLEM_FIG_DIRS,
29     PROBLEM_RESULT_DIRS,
30     RESULT_DIR,
31     ROOT,
32     ensure_dirs,
33     percent,
34     read_data,
35     scenario_driver_assumptions,
36     setup_style,
37 )
38 from 问题一.solve_problem1 import make_problem_one_figures, run_problem_one
39 from 问题二.solve_problem2 import make_problem_two_figures, run_problem_two
40 from 问题三.solve_problem3 import make_problem_three_figures, run_problem_three
41 from 问题四.solve_problem4 import write_policy_report
42
43
44
45
46 def validate_outputs(problem_one: dict[str, pd.DataFrame], problem_two: dict[str, pd.DataFrame], problem_three: dict[str,
47 ↪ pd.DataFrame]) -> None:
48     topsis = problem_one["topsis"]
49     if topsis["TOPSIS 低碳得分"].isna().any() or not topsis["TOPSIS 低碳得分"].between(0, 1).all():
50         raise AssertionError("TOPSIS 得分应非空且位于 0 到 1 之间。")
51     if problem_one["cluster"]["省份"].nunique() != 30:
52         raise AssertionError("K-means 聚类结果应覆盖 30 个省份。")
53     if np.isinf(problem_two["vif"]["VIF"]).any():
54         raise AssertionError("VIF 结果存在无限值。")
55     forecast = problem_three["forecast"]
56     for scenario, group in forecast.groupby("情景"):
57         if not set(range(2026, 2046)).issubset(set(group["年份"])):
58             raise AssertionError(f"{scenario} 未覆盖 2026-2045 完整预测区间。")
59     expected_csvs = {
60         "问题一": ["01_空间差异指标", "02_熵权指标权重", "03_TOPSIS得分排名", "04_聚类K值检验", "05_K4聚类结果", "06_类型均值画像"],
61         "问题二": ["01_OLS系数", "02_拟合优度", "03_VIF共线性检验", "04_拟合值残差", "05_岭回归交叉验证", "06_岭回归标准化系数"],
62         "问题三": [
63             "01_ 全国年度排放",
64             "02_2024 部门排放",
65             "03_ 情景参数",
66             "04_2024-2045 预测序列",
67             "05_ 达峰与减排潜力",
68             "06_STIRPAT 递推系数",
69             "07_ 驱动变量基准值",
70             "08_ 单因素灵敏度分析",
71             "09_STIRPAT 系数灵敏度分析",
72         ],
73     }
74     for problem, stems in expected_csvs.items():
75         for stem in stems:
76             path = PROBLEM_RESULT_DIRS[problem] / f"{stem}.csv"
77             if not path.exists() or path.stat().st_size == 0:
78                 raise AssertionError(f"CSV 结果未正确生成: {path}")
79     expected_figs = {
80         "问题一": [
```

```

80         "01_ 省份 CO2 总量排名图",
81         "02_ 人均 CO2 与碳排放强度散点图",
82         "03_TOPSIS 低碳得分排名图",
83         "04_1_ 资源依赖高排放型 _ 均值画像雷达图",
84         "04_2_ 工业制造高排放型 _ 均值画像雷达图",
85         "04_3_ 中等转型压力型 _ 均值画像雷达图",
86         "04_4_ 经济发达效率型 _ 均值画像雷达图",
87         "05_ 省级 CO2 总量空间分布热力图",
88     ],
89     " 问题二": ["05_STIRPAT 标准化系数图", "06_OLS 拟合值与真实值对比图"],
90     " 问题三": [
91         "07_ 三情景碳排放趋势图",
92         "08_ 三情景峰值与减排潜力对比图",
93         "09_ 单因素灵敏度 _2045 排放影响图",
94         "10_STIRPAT 系数灵敏度 _2045 排放影响图",
95     ],
96 }
97 for problem, stems in expected_figs.items():
98     for stem in stems:
99         path = PROBLEM_FIG_DIRS[problem] / f"{stem}.png"
100         if not path.exists() or path.stat().st_size == 0:
101             raise AssertionError(f" 图片未正确生成: {path}")
102
103
104 def main() -> None:
105     ensure_dirs()
106     setup_style()
107     province, national, carbon = read_data()
108     problem_one = run_problem_one(province)
109     problem_two = run_problem_two(province)
110     problem_three = run_problem_three(carbon, national, province, problem_two)
111     make_problem_one_figures(problem_one)
112     make_problem_two_figures(problem_two)
113     make_problem_three_figures(problem_three)
114     write_policy_report(problem_one, problem_two, problem_three)
115     validate_outputs(problem_one, problem_two, problem_three)
116     print(" 全部结果已按问题分类生成。")
117     print(f" 结果目录: {RESULT_DIR}")
118     print(f" 图片目录: {FIG_DIR}")
119
120
121 if __name__ == "__main__":
122     main()

```

附录 B 公共函数 common.py

```

1  from __future__ import annotations
2
3  import math
4  import warnings
5  from pathlib import Path
6
7  import matplotlib.pyplot as plt
8  import numpy as np
9  import pandas as pd
10 import seaborn as sns
11 from matplotlib import font_manager
12
13
14 warnings.filterwarnings("ignore", category=UserWarning)
15
16 ROOT = Path(__file__).resolve().parents[1]
17 DATA_DIR = ROOT / " 数据"
18 RESULT_DIR = ROOT / "1. 结果"
19 FIG_DIR = ROOT / "2. 图片"
20 CODE_DIR = ROOT / "3. 代码"
21
22 PROBLEM_RESULT_DIRS = {
23     " 问题一": RESULT_DIR / " 问题一",
24     " 问题二": RESULT_DIR / " 问题二",
25     " 问题三": RESULT_DIR / " 问题三",
26     " 问题四": RESULT_DIR / " 问题四",
27     " 汇总": RESULT_DIR / " 汇总",
28 }
29
30 PROBLEM_FIG_DIRS = {
31     " 问题一": FIG_DIR / " 问题一",
32     " 问题二": FIG_DIR / " 问题二",
33     " 问题三": FIG_DIR / " 问题三",
34     " 问题四": FIG_DIR / " 问题四",
35 }
36
37 PROVINCE_FILE = DATA_DIR / "2022 省级综合表.xlsx"
38 NATIONAL_FILE = DATA_DIR / " 全国数据.xlsx"
39 CARBON_FILE = DATA_DIR / " 附件 1: 2019—2025 年全国碳排放数据.csv"
40
41
42 def ensure_dirs() -> None:
43     RESULT_DIR.mkdir(exist_ok=True)
44     FIG_DIR.mkdir(exist_ok=True)

```

```

45     for path in [*PROBLEM_RESULT_DIRS.values(), *PROBLEM_FIG_DIRS.values()]:
46         path.mkdir(parents=True, exist_ok=True)
47
48
49 def setup_style() -> None:
50     sns.set_theme(style="whitegrid", context="notebook")
51     candidates = [
52         "Microsoft YaHei",
53         "SimHei",
54         "Noto Sans CJK SC",
55         "Source Han Sans SC",
56         "Arial Unicode MS",
57     ]
58     available = {f.name for f in font_manager.fontManager.ttflist}
59     for font in candidates:
60         if font in available:
61             plt.rcParams["font.sans-serif"] = [font]
62             break
63     plt.rcParams["axes.unicode_minus"] = False
64     plt.rcParams["figure.dpi"] = 150
65     plt.rcParams["savefig.dpi"] = 300
66
67
68 def save_fig(fig: plt.Figure, output_dir: Path, stem: str) -> None:
69     fig.tight_layout()
70     output_dir.mkdir(parents=True, exist_ok=True)
71     fig.savefig(output_dir / f"{stem}.png", bbox_inches="tight")
72     plt.close(fig)
73
74
75 def read_data() -> tuple[pd.DataFrame, pd.DataFrame, pd.DataFrame]:
76     province = pd.read_excel(PROVINCE_FILE, sheet_name="主表")
77     national = pd.read_excel(NATIONAL_FILE)
78     carbon = pd.read_csv(CARBON_FILE, encoding="utf-8-sig")
79     carbon["Date"] = pd.to_datetime(carbon["Date"])
80     carbon["Year"] = carbon["Date"].dt.year
81     return province, national, carbon
82
83
84 def require_columns(df: pd.DataFrame, columns: list[str]) -> None:
85     missing = [col for col in columns if col not in df.columns]
86     if missing:
87         raise ValueError(f"缺少必要字段: {missing}")
88     if df[columns].isna().any().any():
89         missing_counts = df[columns].isna().sum()
90         raise ValueError(f"关键字段存在缺失值:\n{missing_counts[missing_counts > 0]}")
91
92
93 def gini(values: pd.Series | np.ndarray) -> float:
94     arr = np.sort(np.asarray(values, dtype=float))
95     n = len(arr)
96     if n == 0 or arr.sum() == 0:
97         return np.nan
98     return float((2 * np.arange(1, n + 1) @ arr) / (n * arr.sum()) - (n + 1) / n)
99
100
101 def theil(values: pd.Series | np.ndarray) -> float:
102     arr = np.asarray(values, dtype=float)
103     mean = arr.mean()
104     if mean == 0:
105         return np.nan
106     ratio = arr / mean
107     ratio = ratio[ratio > 0]
108     return float(np.mean(ratio * np.log(ratio)))
109
110
111
112 def write_csv(df: pd.DataFrame, path: Path) -> None:
113     path.parent.mkdir(parents=True, exist_ok=True)
114     df.to_csv(path, index=False, encoding="utf-8-sig")
115
116
117 def write_gitignore() -> None:
118     gitignore = ROOT / ".gitignore"
119     desired = [
120         ".venv/",
121         ".pycache/",
122         "__pycache__/",
123         "*.pyc",
124         ".pytest_cache/",
125         ".mypy_cache/",
126         ".ipynb_checkpoints/",
127         ".*.xlsx",
128         ".*.docx",
129     ]
130     existing = gitignore.read_text(encoding="utf-8").splitlines() if gitignore.exists() else []
131     merged = existing[:]
132     for line in desired:
133         if line not in merged:
134             merged.append(line)
135     gitignore.write_text("\n".join(merged).rstrip() + "\n", encoding="utf-8")
136

```

```

137
138 def percent(value: float) -> str:
139     return f"{value:.1%}"
140
141
142 def scenario_driver_assumptions() -> dict[str, dict[str, float]]:
143     return {
144         " 基准情景": {
145             " 人口增长率 _ 起始": -0.0010,
146             " 人口增长率 _ 末期": -0.0020,
147             " 人均 GDP 增长率 _ 起始": 0.0450,
148             " 人均 GDP 增长率 _ 末期": 0.0260,
149             " 煤炭占比年变化 _ 起始": -0.0040,
150             " 煤炭占比年变化 _ 末期": -0.0100,
151             " 第二产业占比年变化 _ 起始": -0.0005,
152             " 第二产业占比年变化 _ 末期": -0.0020,
153             " 城镇化率年变化 _ 起始": 0.0035,
154             " 城镇化率年变化 _ 末期": 0.0015,
155         },
156         " 低碳情景": {
157             " 人口增长率 _ 起始": -0.0010,
158             " 人口增长率 _ 末期": -0.0020,
159             " 人均 GDP 增长率 _ 起始": 0.0430,
160             " 人均 GDP 增长率 _ 末期": 0.0240,
161             " 煤炭占比年变化 _ 起始": -0.0060,
162             " 煤炭占比年变化 _ 末期": -0.0140,
163             " 第二产业占比年变化 _ 起始": -0.0010,
164             " 第二产业占比年变化 _ 末期": -0.0030,
165             " 城镇化率年变化 _ 起始": 0.0030,
166             " 城镇化率年变化 _ 末期": 0.0010,
167         },
168         " 强化低碳情景": {
169             " 人口增长率 _ 起始": -0.0010,
170             " 人口增长率 _ 末期": -0.0025,
171             " 人均 GDP 增长率 _ 起始": 0.0410,
172             " 人均 GDP 增长率 _ 末期": 0.0220,
173             " 煤炭占比年变化 _ 起始": -0.0080,
174             " 煤炭占比年变化 _ 末期": -0.0180,
175             " 第二产业占比年变化 _ 起始": -0.0020,
176             " 第二产业占比年变化 _ 末期": -0.0050,
177             " 城镇化率年变化 _ 起始": 0.0025,
178             " 城镇化率年变化 _ 末期": 0.0005,
179         },
180     }

```

附录 C 问题一求解脚本 solve_problem1.py

```

1 from __future__ import annotations
2
3 """
4 问题一脚本说明
5
6 算法与模型：
7 - 变异系数、基尼系数、泰尔指数：度量 30 省碳排放空间差异和离散程度。
8 - 熵权 TOPSIS：构建低碳发展综合评价得分，指标包括排放规模、排放效率、能源结构、产业结构和经济发展水平。
9 - K-means 聚类：按标准化后的 7 个指标将省份划分为 4 类，并计算 K=2 至 6 的轮廓系数作为检验。
10
11 使用数据：
12 - 输入：数据/2022 省级综合表.xlsx，工作表“主表”。
13 - 辅助边界：3。代码/assets/china_provinces.geojson，用于绘制省级空间分布热力图。
14 - 输出：1. 结果/问题一/ 下多个 CSV 结果文件。
15 - 输出图片：2. 图片/问题一/01 至 05 号图。
16 """
17
18 import sys
19 from pathlib import Path
20
21 CODE_ROOT = Path(__file__).resolve().parents[1]
22 ROOT = CODE_ROOT.parent
23 sys.pycache_prefix = str(ROOT / ".pycache")
24 sys.path.append(str(CODE_ROOT))
25
26 import json
27 import matplotlib.pyplot as plt
28 from matplotlib.collections import PatchCollection
29 from matplotlib.patheffects import path_effects
30 from matplotlib.patches import Patch
31 from matplotlib.patches import Polygon as MplPolygon
32 import numpy as np
33 import pandas as pd
34 import seaborn as sns
35 from sklearn.cluster import KMeans
36 from sklearn.metrics import silhouette_score
37 from sklearn.preprocessing import StandardScaler
38
39 from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, gini, read_data, require_columns, save_fig, setup_style, theil,
40 ↪ write_csv
41
42 PROBLEM = " 问题一"

```

```

43 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
44 FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
45 CHINA_GEOJSON = CODE_ROOT / "assets" / "china_provinces.geojson"
46
47
48 def normalize_province_name(name: str) -> str:
49     result = str(name).strip()
50     replacements = {
51         " 特别行政区": "",
52         " 维吾尔自治区": "",
53         " 壮族自治区": "",
54         " 回族自治区": "",
55         " 自治区": "",
56         " 省": "",
57         " 市": "",
58     }
59     for old, new in replacements.items():
60         result = result.replace(old, new)
61     return result
62
63
64 def geometry_to_polygons(geometry: dict) -> list[np.ndarray]:
65     polygons = []
66     if geometry["type"] == "Polygon":
67         coordinates = [geometry["coordinates"]]
68     elif geometry["type"] == "MultiPolygon":
69         coordinates = geometry["coordinates"]
70     else:
71         return polygons
72     for polygon in coordinates:
73         if not polygon:
74             continue
75         exterior = np.asarray(polygon[0], dtype=float)
76         if len(exterior) >= 3:
77             polygons.append(exterior)
78     return polygons
79
80
81 def entropy_topsis(df: pd.DataFrame) -> tuple[pd.DataFrame, pd.DataFrame]:
82     indicators = [
83         ("CO2 总量 _Mt", " 负向"),
84         (" 人均 CO2_ 吨每人", " 负向"),
85         (" 碳排放强度 _ 吨每万元 GDP", " 负向"),
86         (" 煤炭相关排放占比", " 负向"),
87         (" 第二产业占比", " 负向"),
88         (" 人均 GDP_ 万元每人", " 正向"),
89         (" 城镇化率 _%", " 正向"),
90     ]
91     matrix = []
92     for col, direction in indicators:
93         x = df[col].astype(float).to_numpy()
94         span = x.max() - x.min()
95         if span == 0:
96             z = np.ones_like(x)
97         elif direction == " 正向":
98             z = (x - x.min()) / span
99         else:
100             z = (x.max() - x) / span
101         matrix.append(z + 1e-12)
102
103     z = np.vstack(matrix).T
104     p = z / z.sum(axis=0)
105     entropy = -(1 / np.log(len(df))) * np.sum(p * np.log(p), axis=0)
106     weights = (1 - entropy) / (1 - entropy).sum()
107     d_pos = np.sqrt(((z - z.max(axis=0)) ** 2 * weights).sum(axis=1))
108     d_neg = np.sqrt(((z - z.min(axis=0)) ** 2 * weights).sum(axis=1))
109     score = d_neg / (d_pos + d_neg)
110
111     weights_df = pd.DataFrame(
112         {
113             " 指标": [col for col, _ in indicators],
114             " 属性": [direction for _, direction in indicators],
115             " 熵值": entropy,
116             " 权重": weights,
117         }
118     )
119     result = df[[" 省份"]] + [col for col, _ in indicators].copy()
120     result["TOPSIS 低碳得分"] = score
121     result["TOPSIS 排名"] = result["TOPSIS 低碳得分"].rank(ascending=False, method="min").astype(int)
122     result[" 低碳等级"] = pd.qcut(
123         result["TOPSIS 低碳得分"].rank(method="first"),
124         q=4,
125         labels=["IV 级 _ 转型压力大", "III 级 _ 中等压力", "II 级 _ 较好", "I 级 _ 优秀"],
126     )
127     return result.sort_values("TOPSIS 排名"), weights_df
128
129
130 def run_problem_one(province: pd.DataFrame) -> dict[str, pd.DataFrame]:
131     key_cols = [
132         "CO2 总量 _Mt",
133         " 人口 _ 万人",
134         "GDP_ 亿元",

```



```

135     " 人均 CO2_ 吨每人",
136     " 碳排放强度 _ 吨每万元 GDP",
137     " 煤炭相关排放占比",
138     " 第二产业占比",
139     " 人均 GDP_ 万元每人",
140     " 城镇化率 _%",
141 ]
142 require_columns(province, [" 省份"] + key_cols)
143 if len(province) != 30:
144     raise ValueError(f" 省级主表应为 30 行, 当前为 {len(province)} 行")
145
146 spatial_cols = ["CO2 总量 _Mt", " 人均 CO2_ 吨每人", " 碳排放强度 _ 吨每万元 GDP", " 煤炭相关排放占比"]
147 spatial_stats = []
148 for col in spatial_cols:
149     values = province[col].astype(float)
150     spatial_stats.append(
151         {
152             " 指标": col,
153             " 均值": values.mean(),
154             " 标准差": values.std(ddof=0),
155             " 变异系数": values.std(ddof=0) / values.mean(),
156             " 基尼系数": gini(values),
157             " 泰尔指数": theil(values),
158             " 最小值": values.min(),
159             " 最大值": values.max(),
160         }
161     )
162 spatial_stats_df = pd.DataFrame(spatial_stats)
163 topsis_df, weights_df = entropy_topsis(province)
164
165 cluster_features = [
166     "CO2 总量 _Mt",
167     " 人均 CO2_ 吨每人",
168     " 碳排放强度 _ 吨每万元 GDP",
169     " 煤炭相关排放占比",
170     " 第二产业占比",
171     " 人均 GDP_ 万元每人",
172     " 城镇化率 _%",
173 ]
174 x = StandardScaler().fit_transform(province[cluster_features])
175 silhouette_df = pd.DataFrame(
176     [
177         {
178             "K": k,
179             " 轮廓系数": silhouette_score(x, KMeans(n_clusters=k, n_init=50, random_state=42).fit_predict(x)),
180         }
181         for k in range(2, 7)
182     ]
183 )
184
185 labels = KMeans(n_clusters=4, n_init=100, random_state=42).fit_predict(x)
186 cluster_df = province[[" 省份"] + cluster_features].copy()
187 cluster_df[" 聚类编号"] = labels
188 profile = cluster_df.groupby(" 聚类编号")[cluster_features].mean()
189 high_resource = profile[" 人均 CO2_ 吨每人"].idxmax()
190 efficient = profile[" 碳排放强度 _ 吨每万元 GDP"].idxmin()
191 remaining = [idx for idx in profile.index if idx not in {high_resource, efficient}]
192 industrial = profile.loc[remaining, "CO2 总量 _Mt"].idxmax()
193 transition = [idx for idx in remaining if idx != industrial][0]
194 cluster_names = {
195     high_resource: " 资源依赖高排放型",
196     industrial: " 工业制造高排放型",
197     transition: " 中等转型压力型",
198     efficient: " 经济发达效率型",
199 }
200 cluster_df[" 类型"] = cluster_df[" 聚类编号"].map(cluster_names)
201 cluster_profile = (
202     cluster_df.groupby(" 类型")[cluster_features]
203     .mean()
204     .reset_index()
205     .sort_values("CO2 总量 _Mt", ascending=False)
206 )
207
208 RESULT_DIR.mkdir(parents=True, exist_ok=True)
209 write_csv(spatial_stats_df, RESULT_DIR / "01_ 空间差异指标.csv")
210 write_csv(weights_df, RESULT_DIR / "02_ 熵权指标权重.csv")
211 write_csv(topsis_df, RESULT_DIR / "03_TOPSIS 得分排名.csv")
212 write_csv(silhouette_df, RESULT_DIR / "04_ 聚类 K 值检验.csv")
213 write_csv(cluster_df.sort_values([" 类型", " 省份"]), RESULT_DIR / "05_K4 聚类结果.csv")
214 write_csv(cluster_profile, RESULT_DIR / "06_ 类型均值画像.csv")
215
216 return {
217     "spatial_stats": spatial_stats_df,
218     "topsis": topsis_df,
219     "weights": weights_df,
220     "silhouette": silhouette_df,
221     "cluster": cluster_df,
222     "cluster_profile": cluster_profile,
223 }
224
225
226 def make_problem_one_figures(problem_one: dict[str, pd.DataFrame]) -> None:

```

```

227 topsis = problem_one["topsis"].copy()
228 cluster = problem_one["cluster"].copy()
229 province = topsis.merge(cluster[["省份", "类型"]], on="省份", how="left")
230
231 top_total = province.sort_values("CO2 总量 _Mt", ascending=True)
232 fig, ax = plt.subplots(figsize=(9, 8))
233 ax.barh(top_total["省份"], top_total["CO2 总量 _Mt"], color=sns.color_palette("viridis", n_colors=len(top_total)))
234 ax.set_title("2022 年 30 省 CO2 排放总量排名")
235 ax.set_xlabel("CO2 排放总量 (Mt) ")
236 ax.set_ylabel("")
237 save_fig(fig, FIG_DIR, "01_ 省份 CO2 总量排名图")
238
239 fig, ax = plt.subplots(figsize=(9, 6))
240 scatter_data = province.rename(columns={"CO2 总量 _Mt": "CO² 总量 (Mt)"})
241 sns.scatterplot(
242     data=scatter_data,
243     x="人均 CO2_ 吨每人",
244     y="碳排放强度 _ 吨每万元 GDP",
245     hue="类型",
246     size="CO² 总量 (Mt) ",
247     sizes=(120, 900),
248     alpha=0.8,
249     ax=ax,
250 )
251 label_offsets = {
252     "内蒙古": (-54, -2),
253     "宁夏": (18, 6),
254     "山西": (26, -4),
255     "新疆": (24, 8),
256     "河北": (30, -8),
257 }
258 label_data = province[province["类型"].eq("资源依赖高排放型")]
259 for _, row in label_data.iterrows():
260     offset = label_offsets.get(row["省份"], (18, 6))
261     ax.annotate(
262         row["省份"],
263         xy=(row["人均 CO2_ 吨每人"], row["碳排放强度 _ 吨每万元 GDP"]),
264         xytext=offset,
265         textcoords="offset points",
266         ha="right" if offset[0] < 0 else "left",
267         va="center",
268         fontsize=9,
269         annotation_clip=False,
270     )
271 ax.set_title("人均 CO2 与碳排放强度关系")
272 ax.set_xlabel("人均 CO2 (吨/人)")
273 ax.set_ylabel("碳排放强度 (吨/万元 GDP)")
274 handles, labels = ax.get_legend_handles_labels()
275 ax.legend_.remove()
276 size_header = "CO² 总量 (Mt) "
277 size_start = labels.index(size_header)
278 type_handles = handles[1:size_start]
279 type_labels = labels[1:size_start]
280 size_handles = handles[size_start + 1 :]
281 size_labels = labels[size_start + 1 :]
282 type_legend = ax.legend(type_handles, type_labels, title="类型", loc="upper left", frameon=True)
283 ax.add_artist(type_legend)
284 ax.legend(size_handles, size_labels, title=size_header, loc="lower right", frameon=True)
285 save_fig(fig, FIG_DIR, "02_ 人均 CO2 与碳排放强度散点图")
286
287 if not CHINA_GEOJSON.exists():
288     raise FileNotFoundError(f"缺少中国省级边界文件: {CHINA_GEOJSON}")
289 geojson = json.loads(CHINA_GEOJSON.read_text(encoding="utf-8"))
290 value_map = dict(zip(province["省份"].map(normalize_province_name), province["CO2 总量 _Mt"]))
291 values = np.array(list(value_map.values()), dtype=float)
292 norm = plt.Normalize(values.min(), values.max())
293 cmap = plt.get_cmap("YlOrRd")
294
295 fig, ax = plt.subplots(figsize=(14, 9.2))
296 missing_patches = []
297 data_patches = []
298 data_colors = []
299 label_points = []
300 callout_positions = {
301     "北京": (124.4, 40.2),
302     "天津": (124.4, 38.4),
303     "上海": (124.2, 31.2),
304     "海南": (113.3, 18.7),
305 }
306 text_effect = [path_effects.withStroke(linewidth=3.8, foreground="white")]
307 for feature in geojson["features"]:
308     raw_name = feature.get("properties", {}).get("name", "")
309     if not raw_name:
310         continue
311     province_name = normalize_province_name(raw_name)
312     polygons = geometry_to_polygons(feature["geometry"])
313     if province_name in value_map:
314         color = cmap(norm(value_map[province_name]))
315         for polygon in polygons:
316             data_patches.append(MplPolygon(polygon, closed=True))
317             data_colors.append(color)
318     center = feature.get("properties", {}).get("centroid") or feature.get("properties", {}).get("center")

```

```

319         if center:
320             label_points.append((province_name, center[0], center[1], value_map[province_name]))
321         else:
322             for polygon in polygons:
323                 missing_patches.append(MplPolygon(polygon, closed=True))
324
325     if missing_patches:
326         ax.add_collection(PatchCollection(missing_patches, facecolor="#E5E7EB", edgecolor="white", linewidth=0.7, zorder=1))
327     if data_patches:
328         ax.add_collection(PatchCollection(data_patches, facecolor=data_colors, edgecolor="white", linewidth=0.8, zorder=2))
329     for name, x_coord, y_coord, value in label_points:
330         label = f"{name}\n{value:.1f}"
331         if name in callout_positions:
332             text_x, text_y = callout_positions[name]
333             ax.plot([x_coord, text_x], [y_coord, text_y], color="#111827", linewidth=0.8, zorder=3)
334             ax.scatter([x_coord], [y_coord], s=10, color="#111827", zorder=4)
335             text = ax.text(
336                 text_x,
337                 text_y,
338                 label,
339                 fontsize=8.4,
340                 ha="left",
341                 va="center",
342                 color="#111827",
343                 linespacing=1.12,
344                 zorder=5,
345             )
346         else:
347             text = ax.text(
348                 x_coord,
349                 y_coord,
350                 label,
351                 fontsize=8.2,
352                 ha="center",
353                 va="center",
354                 color="#111827",
355                 linespacing=1.12,
356                 zorder=4,
357             )
358         text.set_path_effects(text_effect)
359     sm = plt.cm.ScalarMappable(norm=norm, cmap=cmap)
360     sm.set_array([])
361     cbar = fig.colorbar(sm, ax=ax, fraction=0.032, pad=0.02)
362     cbar.set_label("CO2 总量 (Mt) ")
363     ax.legend(handles=[Patch(facecolor="#E5E7EB", edgecolor="white", label="无数据或未纳入30省样本")], loc="lower left",
364               ↪ frameon=True)
365     ax.set_title("2022 年省级 CO2 排放总量空间分布")
366     ax.set_xlim(72, 136)
367     ax.set_ylim(17, 55)
368     ax.set_aspect("equal")
369     ax.axis("off")
370     save_fig(fig, FIG_DIR, "05_ 省级 CO2 总量空间分布热力图")
371
372     topsis_sorted = topsis.sort_values("TOPSIS 低碳得分", ascending=True)
373     fig, ax = plt.subplots(figsize=(9, 8))
374     ax.barh(topsis_sorted["省份"], topsis_sorted["TOPSIS 低碳得分"], color=sns.color_palette("crest", len(topsis_sorted)))
375     ax.set_title("  熵权 TOPSIS 低碳发展得分排名")
376     ax.set_xlabel("TOPSIS 低碳得分")
377     ax.set_ylabel("")
378     save_fig(fig, FIG_DIR, "03_TOPSIS 低碳得分排名图")
379
380     old_radar = FIG_DIR / "04_ 聚类类型均值画像雷达图.png"
381     if old_radar.exists():
382         old_radar.unlink()
383
384     profile = problem_one["cluster_profile"].copy()
385     radar_cols = ["CO2总量_Mt", "人均CO2_吨每人", "碳排放强度_吨每万元GDP", "煤炭相关排放占比", "第二产业占比", "人均GDP_万元每人",
386                 ↪ "城镇化率_%"]
387     scaled = profile.copy()
388     for col in radar_cols:
389         mn, mx = cluster[col].min(), cluster[col].max()
390         scaled[col] = (profile[col] - mn) / (mx - mn) if mx > mn else 0
391     labels = ["总量", "人均", "强度", "煤炭", "二产", "人均 GDP", "城镇化"]
392     angles = np.linspace(0, 2 * np.pi, len(labels), endpoint=False)
393     polygon_angles = np.r_[angles, angles[0]]
394     palette = dict(zip(scaled["类型"], sns.color_palette("deep", n_colors=len(scaled))))
395
396     for idx, (_, row) in enumerate(scaled.iterrows(), start=1):
397         values = np.array([row[col] for col in radar_cols], dtype=float)
398         closed_values = np.r_[values, values[0]]
399         color = palette[row["类型"]]
400
401         fig = plt.figure(figsize=(7, 7))
402         ax = fig.add_subplot(111, polar=True)
403         ax.grid(False)
404         ax.spines["polar"].set_visible(False)
405         ax.set_ylim(0, 1)
406         ax.set_yticks([])
407         ax.set_theta_offset(np.pi / 2)
408         ax.set_theta_direction(-1)
409
410         # Draw polygon gridlines and radial spokes manually, replacing circular grids.

```

```

409     for radius in [0.2, 0.4, 0.6, 0.8, 1.0]:
410         ax.plot(polygon_angles, [radius] * len(polygon_angles), color="#D1D5DB", linewidth=1)
411     for angle in angles:
412         ax.plot([angle, angle], [0, 1], color="#D1D5DB", linewidth=1)
413
414     ax.plot(polygon_angles, closed_values, color=color, linewidth=2.6)
415     ax.fill(polygon_angles, closed_values, color=color, alpha=0.20)
416     ax.scatter(angles, values, color=color, s=42, zorder=3)
417     ax.set_xticks(angles)
418     ax.set_xticklabels(labels, fontsize=11)
419     ax.set_title(row[" 类型"], fontsize=17, pad=22)
420     save_fig(fig, FIG_DIR, f"04_{idx}_{row['类型']}_ 均值画像雷达图")
421
422
423 def main() -> None:
424     setup_style()
425     province, _, _ = read_data()
426     result = run_problem_one(province)
427     make_problem_one_figures(result)
428     print(f" 问题一结果已生成: {RESULT_DIR}")
429     print(f" 问题一图片已生成: {FIG_DIR}")
430
431
432 if __name__ == "__main__":
433     main()

```

附录 D 问题二求解脚本 solve_problem2.py

```

1  from __future__ import annotations
2
3  """
4  问题二脚本说明
5
6  算法与模型:
7  - STIRPAT 横截面回归:  $\ln(CO_2)=a+b_1 \ln(P)+b_2 \ln(A)+b_3 \text{Coal}+b_4 \text{IS}+b_5 U+e$ 。
8  - OLS 显著性检验: 输出系数、标准误、t 值、p 值、R2、调整 R2。
9  - VIF 共线性检验: 判断解释变量是否存在严重多重共线性。
10 - 岭回归留一交叉验证: 用正则化模型检验驱动因素排序和预测稳健性。
11
12 使用数据:
13 - 输入: 数据/2022 省级综合表.xlsx, 工作表“主表”。
14 - 输出: 1. 结果/问题二/ 下多个 CSV 结果文件。
15 - 输出图片: 2. 图片/问题二/05_STIRPAT 标准化系数图、06_OLS 拟合值与真实值对比图。
16 """
17
18 import sys
19 from pathlib import Path
20
21 CODE_ROOT = Path(__file__).resolve().parents[1]
22 ROOT = CODE_ROOT.parent
23 sys.pycache_prefix = str(ROOT / ".pycache")
24 sys.path.append(str(CODE_ROOT))
25
26 import matplotlib.pyplot as plt
27 from matplotlib.lines import Line2D
28 import numpy as np
29 import pandas as pd
30 import seaborn as sns
31 import statsmodels.api as sm
32 from sklearn.linear_model import RidgeCV
33 from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score
34 from sklearn.model_selection import LeaveOneOut, cross_val_predict
35 from sklearn.preprocessing import StandardScaler
36 from statsmodels.stats.outliers_influence import variance_inflation_factor
37
38 from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, read_data, save_fig, setup_style, write_csv
39
40
41 PROBLEM = " 问题二"
42 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
43 FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
44
45
46 def run_problem_two(province: pd.DataFrame) -> dict[str, pd.DataFrame]:
47     df = province.copy()
48     df["lnC"] = np.log(df["CO2 总量 _Mt"])
49     df["lnP"] = np.log(df[" 人口 _ 万人"])
50     df["lnA"] = np.log(df[" 人均 GDP_ 万元每人"])
51     df[" 城镇化率"] = df[" 城镇化率 _%"] / 100
52     predictors = ["lnP", "lnA", " 煤炭相关排放占比", " 第二产业占比", " 城镇化率"]
53     x = sm.add_constant(df[predictors])
54     y = df["lnC"]
55     ols = sm.OLS(y, x).fit()
56
57     ols_df = pd.DataFrame(
58         {
59             " 变量": ols.params.index,
60             " 系数": ols.params.values,
61             " 标准误": ols.bse.values,
62             "t 值": ols.tvalues.values,

```

```

63         "p 值": ols.pvalues.values,
64     }
65 )
66 fit_df = pd.DataFrame(
67     {
68         "指标": ["R2", "调整 R2", "F 统计量", "F 检验 p 值", "AIC", "BIC", "RMSE_log", "MAE_log"],
69         "数值": [
70             ols.rsquared,
71             ols.rsquared_adj,
72             ols.fvalue,
73             ols.f_pvalue,
74             ols.aic,
75             ols.bic,
76             mean_squared_error(y, ols.fittedvalues) ** 0.5,
77             mean_absolute_error(y, ols.fittedvalues),
78         ],
79     }
80 )
81 vif_df = pd.DataFrame(
82     {
83         "变量": predictors,
84         "VIF": [variance_inflation_factor(x.values, i + 1) for i in range(len(predictors))],
85     }
86 )
87 fitted_df = pd.DataFrame(
88     {
89         "省份": df["省份"],
90         "实际 lnCO2": y,
91         "拟合 lnCO2": ols.fittedvalues,
92         "残差": ols.resid,
93         "实际 CO2_Mt": df["CO2 总量_Mt"],
94         "拟合 CO2_Mt": np.exp(ols.fittedvalues),
95     }
96 )
97 fitted_df["绝对误差_Mt"] = (fitted_df["拟合 CO2_Mt"] - fitted_df["实际 CO2_Mt"]).abs()
98 fitted_df["误差方向"] = np.where(fitted_df["拟合 CO2_Mt"] > fitted_df["实际 CO2_Mt"], "模型高估", "模型低估")
99
100 scaler = StandardScaler()
101 xs = scaler.fit_transform(df[predictors])
102 ridge_cv = RidgeCV(alphas=np.logspace(-3, 3, 100), cv=LeaveOneOut())
103 ridge_pred = cross_val_predict(ridge_cv, xs, y, cv=LeaveOneOut())
104 ridge_cv.fit(xs, y)
105 ridge_coef_df = pd.DataFrame(
106     {
107         "变量": predictors,
108         "标准化岭回归系数": ridge_cv.coef_,
109         "绝对值排序": np.abs(ridge_cv.coef_).argsort()[::-1].argsort() + 1,
110     }
111 ).sort_values("绝对值排序")
112 ridge_metrics_df = pd.DataFrame(
113     {
114         "指标": ["最优 alpha", "LOOCV_R2", "LOOCV_RMSE_log", "LOOCV_MAE_log"],
115         "数值": [
116             ridge_cv.alpha_,
117             r2_score(y, ridge_pred),
118             mean_squared_error(y, ridge_pred) ** 0.5,
119             mean_absolute_error(y, ridge_pred),
120         ],
121     }
122 )
123
124 RESULT_DIR.mkdir(parents=True, exist_ok=True)
125 write_csv(ols_df, RESULT_DIR / "01_OLS 系数.csv")
126 write_csv(fit_df, RESULT_DIR / "02_拟合优度.csv")
127 write_csv(vif_df, RESULT_DIR / "03_VIF 共线性检验.csv")
128 write_csv(fitted_df, RESULT_DIR / "04_拟合值残差.csv")
129 write_csv(ridge_metrics_df, RESULT_DIR / "05_岭回归交叉验证.csv")
130 write_csv(ridge_coef_df, RESULT_DIR / "06_岭回归标准化系数.csv")
131
132 return {
133     "ols_coef": ols_df,
134     "fit": fit_df,
135     "vif": vif_df,
136     "fitted": fitted_df,
137     "ridge_metrics": ridge_metrics_df,
138     "ridge_coef": ridge_coef_df,
139 }
140
141
142 def make_problem_two_figures(problem_two: dict[str, pd.DataFrame]) -> None:
143     ridge_coef = problem_two["ridge_coef"].sort_values("标准化岭回归系数")
144     variable_labels = {
145         "lnP": "人口规模 (lnP)",
146         "lnA": "经济发展水平 (lnA)",
147         "煤炭相关排放占比": "煤炭相关排放占比",
148         "第二产业占比": "第二产业占比",
149         "城镇化率": "城镇化率",
150     }
151     variable_colors = {
152         "lnP": "#2563EB",
153         "lnA": "#059669",
154         "煤炭相关排放占比": "#DC2626",

```

```

155     " 第二产业占比": "#D97706",
156     " 城镇化率": "#7C3AED",
157 }
158 ridge_coef[" 变量说明"] = ridge_coef[" 变量"].map(variable_labels).fillna(ridge_coef[" 变量"])
159 bar_colors = ridge_coef[" 变量"].map(variable_colors).fillna("#4B5563")
160 fig, ax = plt.subplots(figsize=(8, 5))
161 ax.barh(ridge_coef[" 变量说明"], ridge_coef[" 标准化岭回归系数"], color=bar_colors)
162 ax.axvline(0, color="#333333", linewidth=1)
163 ax.set_title("STIRPAT 岭回归标准化系数")
164 ax.set_xlabel(" 标准化系数")
165 ax.set_ylabel("")
166 save_fig(fig, FIG_DIR, "05_STIRPAT 标准化系数图")
167
168 fitted = problem_two["fitted"].copy()
169 fitted[" 误差_Mt"] = fitted[" 拟合 CO2_Mt"] - fitted[" 实际 CO2_Mt"]
170 lim_min = min(fitted[" 实际 CO2_Mt"].min(), fitted[" 拟合 CO2_Mt"].min()) * 0.88
171 lim_max = max(fitted[" 实际 CO2_Mt"].max(), fitted[" 拟合 CO2_Mt"].max()) * 1.08
172 plot_data = fitted.rename(
173     columns={
174         " 实际 CO2_Mt": " 实际 CO2 排放量 (Mt)",
175         " 拟合 CO2_Mt": " 拟合 CO2 排放量 (Mt)",
176         " 绝对误差_Mt": " 绝对误差 (Mt)",
177     }
178 )
179 plot_data[" 误差方向"] = plot_data[" 误差方向"].map({" 模型高估": " 高估", " 模型低估": " 低估"}).fillna(plot_data[" 误差方向"])
180 error_palette = {" 高估": "#C2410C", " 低估": "#2563EB"}
181
182 fig, ax = plt.subplots(figsize=(8.5, 7))
183 sns.scatterplot(
184     data=plot_data,
185     x=" 实际 CO2 排放量 (Mt)",
186     y=" 拟合 CO2 排放量 (Mt)",
187     hue=" 误差方向",
188     size=" 绝对误差 (Mt)",
189     sizes=(50, 280),
190     palette=error_palette,
191     alpha=0.82,
192     legend=False,
193     ax=ax,
194 )
195 ax.plot([lim_min, lim_max], [lim_min, lim_max], "--", color="#333333", linewidth=1.5)
196 direction_handles = [
197     Line2D([0], [0], marker="o", linestyle="", color=color, label=label, markersize=8, alpha=0.82)
198     for label, color in error_palette.items()
199 ]
200 direction_legend = ax.legend(direction_handles, [" 高估", " 低估"], title=" 误差方向", loc="upper left", frameon=True)
201 ax.add_artist(direction_legend)
202 size_values = [150, 300, 450, 600]
203 size_handles = [
204     Line2D([0], [0], marker="o", linestyle="", color="#595959", label=str(value), markersize=np.sqrt(value / 600) * 16)
205     for value in size_values
206 ]
207 line_handle = Line2D([0], [0], linestyle="--", color="#333333", linewidth=1.5, label=" 完全拟合线")
208 ax.legend(
209     size_handles + [line_handle],
210     [str(value) for value in size_values] + [" 完全拟合线"],
211     title=" 绝对误差 (Mt)",
212     loc="lower right",
213     frameon=True,
214 )
215 ax.set_xlim(lim_min, lim_max)
216 ax.set_ylim(lim_min, lim_max)
217 ax.set_title("STIRPAT 模型实际值与拟合值对比及误差说明")
218 ax.set_xlabel(" 实际 CO2 排放量 (Mt)")
219 ax.set_ylabel(" 模型拟合 CO2 排放量 (Mt)")
220 save_fig(fig, FIG_DIR, "06_OLS 拟合值与真实值对比图")
221
222
223 def main() -> None:
224     setup_style()
225     province, _, _ = read_data()
226     result = run_problem_two(province)
227     make_problem_two_figures(result)
228     print(f" 问题二结果已生成: {RESULT_DIR}")
229     print(f" 问题二图片已生成: {FIG_DIR}")
230
231
232 if __name__ == "__main__":
233     main()

```

附录 E 问题三求解脚本 solve_problem3.py

```

1 from __future__ import annotations
2
3 """
4 问题三脚本说明
5
6 算法与模型:
7 - 全国时间趋势统计: 只使用附件 1 中 Sector=Total 的全国排放总量, 避免 Total 与部门排放重复相加。
8 - STIRPAT 系数递推预测: 使用问题二 OLS 系数, 按驱动变量变化推算全国排放相对变化。

```

```

9 - 达峰研判：逐情景寻找预测序列最大值，输出达峰年份、峰值水平、碳排放强度和减排潜力。
10
11 使用数据：
12 - 输入：数据/附件 1：2019—2025 年全国碳排放数据.csv、数据/全国数据.xlsx、问题二 STIRPAT 系数。
13 - 输出：1. 结果/问题三/ 下多个 CSV 结果文件。
14 - 输出图片：2. 图片/问题三/07_ 三情景碳排放趋势图、08_ 三情景峰值与减排潜力对比图。
15 """
16
17 import sys
18 from pathlib import Path
19
20 CODE_ROOT = Path(__file__).resolve().parents[1]
21 ROOT = CODE_ROOT.parent
22 sys.pycache_prefix = str(ROOT / ".pycache")
23 sys.path.append(str(CODE_ROOT))
24
25 import matplotlib.pyplot as plt
26 import numpy as np
27 import pandas as pd
28 import seaborn as sns
29
30 from common import PROBLEM_FIG_DIRS, PROBLEM_RESULT_DIRS, read_data, save_fig, scenario_driver_assumptions, setup_style,
31 ↪ write_csv
32 from 问题二.solve_problem2 import run_problem_two
33
34 PROBLEM = " 问题三"
35 RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
36 FIG_DIR = PROBLEM_FIG_DIRS[PROBLEM]
37
38
39 def _annual_rate(start: float, end: float, year: int, first_year: int = 2025, last_year: int = 2045) -> float:
40     if last_year == first_year:
41         return start
42     ratio = (year - first_year) / (last_year - first_year)
43     return start + (end - start) * ratio
44
45
46 def _prepare_national_base(national: pd.DataFrame, province: pd.DataFrame, base_year: int) -> tuple[dict[str, float],
47 ↪ pd.DataFrame]:
48     national_std = national.rename(
49         columns={
50             "年 份": " 年份",
51             "GDP (亿元)": "GDP_ 亿元",
52             "人 口 (万人)": "人 口 _ 万人",
53             "能源消费总量 (万吨标准煤)": "能源消费总量 _ 万吨标准煤",
54             "煤炭占比": "煤炭占比 _%",
55         }
56     ).copy()
57     national_std["人均 GDP_ 万元每人"] = national_std["GDP_ 亿元"] / national_std["人 口 _ 万人"]
58     national_std["煤炭占比"] = national_std["煤炭占比 _%"] / 100
59     base = national_std[national_std[" 年份"].eq(base_year)]
60     if base.empty:
61         raise ValueError(f" 全国数据缺少 {base_year} 年，无法确定问题三预测基准。")
62     base_row = base.iloc[0]
63     industry_base = province[" 第二产业 _ 亿元"].sum() / province["GDP_ 亿元"].sum()
64     urban_base = (province[" 城镇化率 _%"] * province["人 口 _ 万人"]).sum() / province["人 口 _ 万人"].sum() / 100
65     base_values = {
66         " 年份": base_year,
67         "人 口 _ 万人": float(base_row["人 口 _ 万人"]),
68         "GDP_ 亿元": float(base_row["GDP_ 亿元"]),
69         "人均 GDP_ 万元每人": float(base_row["人均 GDP_ 万元每人"]),
70         "煤炭相关排放占比": float(base_row["煤炭占比"]),
71         "第二产业占比": float(industry_base),
72         "城镇化率": float(urban_base),
73     }
74     base_table = pd.DataFrame(
75         [
76             {" 变量": "人 口 _ 万人", " 基准值": base_values["人 口 _ 万人"], " 来源": " 全国数据.xlsx 2024 年人口"},
77             {" 变量": "GDP_ 亿元", " 基准值": base_values["GDP_ 亿元"], " 来源": " 全国数据.xlsx 2024 年 GDP"},
78             {" 变量": "人均 GDP_ 万元每人", " 基准值": base_values["人均 GDP_ 万元每人"], " 来源": "GDP/人口"},
79             {" 变量": "煤炭相关排放占比", " 基准值": base_values["煤炭相关排放占比"], " 来源": " 全国数据.xlsx 2024 年煤炭占比"},
80             {" 变量": "第二产业占比", " 基准值": base_values["第二产业占比"], " 来源": "2022 省级综合表按 GDP 聚合"},
81             {" 变量": "城镇化率", " 基准值": base_values["城镇化率"], " 来源": "2022 省级综合表按人口加权"},
82         ]
83     )
84     return base_values, base_table
85
86 def _simulate_forecast(
87     base_emission: float,
88     base_values: dict[str, float],
89     coeff: dict[str, float],
90     assumptions: dict[str, dict[str, float]],
91     base_year: int = 2024,
92 ) -> tuple[pd.DataFrame, pd.DataFrame]:
93     rows = []
94     param_rows = []
95     for scenario, params in assumptions.items():
96         emission = base_emission
97         population = base_values["人 口 _ 万人"]
98         affluence = base_values["人均 GDP_ 万元每人"]

```

```

99     coal_share = base_values[" 煤炭相关排放占比"]
100     industry_share = base_values[" 第二产业占比"]
101     urban_rate = base_values[" 城镇化率"]
102     gdp = population * affluence
103     rows.append(
104         {
105             " 情景": scenario,
106             " 年份": base_year,
107             " 预测 CO2_Mt": emission,
108             " 预测 GDP_亿元": gdp,
109             " 碳排放强度 _ 吨每万元 GDP": emission / gdp * 100,
110             " 人口 _ 万人": population,
111             " 人均 GDP_万元每人": affluence,
112             " 煤炭相关排放占比": coal_share,
113             " 第二产业占比": industry_share,
114             " 城镇化率": urban_rate,
115             "STIRPAT 对数增长项": 0.0,
116         }
117     )
118     for year in range(2025, 2046):
119         g_population = _annual_rate(params[" 人口增长率 _ 起始"], params[" 人口增长率 _ 末期"], year)
120         g_affluence = _annual_rate(params[" 人均 GDP 增长率 _ 起始"], params[" 人均 GDP 增长率 _ 末期"], year)
121         d_coal = _annual_rate(params[" 煤炭占比年变化 _ 起始"], params[" 煤炭占比年变化 _ 末期"], year)
122         d_industry = _annual_rate(params[" 第二产业占比年变化 _ 起始"], params[" 第二产业占比年变化 _ 末期"], year)
123         d_urban = _annual_rate(params[" 城镇化率年变化 _ 起始"], params[" 城镇化率年变化 _ 末期"], year)
124         prev_population = population
125         prev_affluence = affluence
126         prev_coal = coal_share
127         prev_industry = industry_share
128         prev_urban = urban_rate
129         population *= 1 + g_population
130         affluence *= 1 + g_affluence
131         coal_share = float(np.clip(coal_share + d_coal, 0.05, 0.95))
132         industry_share = float(np.clip(industry_share + d_industry, 0.20, 0.60))
133         urban_rate = float(np.clip(urban_rate + d_urban, 0.45, 0.90))
134         d_ln_population = np.log(population / prev_population)
135         d_ln_affluence = np.log(affluence / prev_affluence)
136         d_coal_actual = coal_share - prev_coal
137         d_industry_actual = industry_share - prev_industry
138         d_urban_actual = urban_rate - prev_urban
139         log_growth = (
140             coeff["lnP"] * d_ln_population
141             + coeff["lnA"] * d_ln_affluence
142             + coeff[" 煤炭相关排放占比"] * d_coal_actual
143             + coeff[" 第二产业占比"] * d_industry_actual
144             + coeff[" 城镇化率"] * d_urban_actual
145         )
146         emission *= float(np.exp(log_growth))
147         gdp = population * affluence
148         param_rows.append(
149             {
150                 " 情景": scenario,
151                 " 年份": year,
152                 " 人口增长率": g_population,
153                 " 人均 GDP 增长率": g_affluence,
154                 " 煤炭占比年变化": d_coal_actual,
155                 " 第二产业占比年变化": d_industry_actual,
156                 " 城镇化率年变化": d_urban_actual,
157                 "DeltaInP": d_ln_population,
158                 "DeltaInA": d_ln_affluence,
159                 "STIRPAT 对数增长项": log_growth,
160                 " 排放增长率": np.exp(log_growth) - 1,
161             }
162         )
163     rows.append(
164         {
165             " 情景": scenario,
166             " 年份": year,
167             " 预测 CO2_Mt": emission,
168             " 预测 GDP_亿元": gdp,
169             " 碳排放强度 _ 吨每万元 GDP": emission / gdp * 100,
170             " 人口 _ 万人": population,
171             " 人均 GDP_万元每人": affluence,
172             " 煤炭相关排放占比": coal_share,
173             " 第二产业占比": industry_share,
174             " 城镇化率": urban_rate,
175             "STIRPAT 对数增长项": log_growth,
176         }
177     )
178     return pd.DataFrame(rows), pd.DataFrame(param_rows)
179
180
181 def _summarize_peak(forecast_df: pd.DataFrame, scenario_order: list[str], base_year: int = 2024) -> pd.DataFrame:
182     peak_rows = []
183     for scenario, group in forecast_df.groupby(" 情景"):
184         group = group.sort_values(" 年份")
185         idx = group[" 预测 CO2_Mt"].idxmax()
186         peak_year = int(group.loc[idx, " 年份"])
187         peak_value = float(group.loc[idx, " 预测 CO2_Mt"])
188         base_intensity = float(group.loc[group[" 年份"].eq(base_year), " 碳排放强度 _ 吨每万元 GDP"].iloc[0])
189         intensity_2045 = float(group.loc[group[" 年份"].eq(2045), " 碳排放强度 _ 吨每万元 GDP"].iloc[0])
190         peak_rows.append(

```



```

191     {
192         "情景": scenario,
193         "达峰年份": peak_year,
194         "峰值_Mt": peak_value,
195         "2030 年_Mt": float(group.loc[group["年份"].eq(2030), "预测 CO2_Mt"].iloc[0]),
196         "2035 年_Mt": float(group.loc[group["年份"].eq(2035), "预测 CO2_Mt"].iloc[0]),
197         "2045 年_Mt": float(group.loc[group["年份"].eq(2045), "预测 CO2_Mt"].iloc[0]),
198         "2045 较峰值下降比例": 1 - float(group.loc[group["年份"].eq(2045), "预测 CO2_Mt"].iloc[0]) / peak_value,
199         "2030 年强度_吨每万元 GDP": float(group.loc[group["年份"].eq(2030), "碳排放强度_吨每万元 GDP"].iloc[0]),
200         "2045 年强度_吨每万元 GDP": intensity_2045,
201         "2045 较 2024 强度下降比例": 1 - intensity_2045 / base_intensity,
202     }
203 )
204 peak_df = pd.DataFrame(peak_rows)
205 peak_df["情景"] = pd.Categorical(peak_df["情景"], categories=scenario_order, ordered=True)
206 peak_df = peak_df.sort_values("情景").reset_index(drop=True)
207 peak_df["情景"] = peak_df["情景"].astype(str)
208 return peak_df
209
210
211 def _copy_assumptions(assumptions: dict[str, dict[str, float]]) -> dict[str, dict[str, float]]:
212     return {scenario: values.copy() for scenario, values in assumptions.items()}
213
214
215 def _single_factor_sensitivity(
216     base_emission: float,
217     base_values: dict[str, float],
218     coeff: dict[str, float],
219     assumptions: dict[str, dict[str, float]],
220     baseline_peak: pd.DataFrame,
221 ) -> pd.DataFrame:
222     variable_keys = {
223         "人口增长率": ["人口增长率_起始", "人口增长率_末期"],
224         "人均 GDP 增长率": ["人均 GDP 增长率_起始", "人均 GDP 增长率_末期"],
225         "煤炭占比下降速度": ["煤炭占比年变化_起始", "煤炭占比年变化_末期"],
226         "第二产业占比下降速度": ["第二产业占比年变化_起始", "第二产业占比年变化_末期"],
227         "城镇化率年变化": ["城镇化率年变化_起始", "城镇化率年变化_末期"],
228     }
229     perturbations = [-0.20, -0.10, 0.10, 0.20]
230     baseline = baseline_peak.set_index("情景")
231     rows = []
232     scenario_order = list(assumptions.keys())
233     for scenario in scenario_order:
234         for variable, keys in variable_keys.items():
235             for perturbation in perturbations:
236                 perturbed = _copy_assumptions(assumptions)
237                 for key in keys:
238                     perturbed[scenario][key] *= 1 + perturbation
239                 forecast_df, _ = _simulate_forecast(base_emission, base_values, coeff, perturbed)
240                 peak = _summarize_peak(forecast_df, scenario_order)
241                 result = peak.set_index("情景").loc[scenario]
242                 base = baseline.loc[scenario]
243                 rows.append(
244                     {
245                         "情景": scenario,
246                         "扰动变量": variable,
247                         "扰动比例": perturbation,
248                         "达峰年份": result["达峰年份"],
249                         "达峰年份变化": result["达峰年份"] - base["达峰年份"],
250                         "峰值_Mt": result["峰值_Mt"],
251                         "峰值变化_Mt": result["峰值_Mt"] - base["峰值_Mt"],
252                         "2045 年_Mt": result["2045 年_Mt"],
253                         "2045 变化_Mt": result["2045 年_Mt"] - base["2045 年_Mt"],
254                         "2045 较峰值下降比例": result["2045 较峰值下降比例"],
255                     }
256                 )
257     return pd.DataFrame(rows)
258
259
260 def _coefficient_sensitivity(
261     base_emission: float,
262     base_values: dict[str, float],
263     coeff: dict[str, float],
264     assumptions: dict[str, dict[str, float]],
265     baseline_peak: pd.DataFrame,
266 ) -> pd.DataFrame:
267     variable_labels = {
268         "lnP": "人口规模系数",
269         "lnA": "人均 GDP 系数",
270         "煤炭相关排放占比": "煤炭占比系数",
271         "第二产业占比": "第二产业占比系数",
272         "城镇化率": "城镇化率系数",
273     }
274     perturbations = [-0.10, 0.10]
275     baseline = baseline_peak.set_index("情景")
276     rows = []
277     scenario_order = list(assumptions.keys())
278     for coeff_name, label in variable_labels.items():
279         for perturbation in perturbations:
280             perturbed_coeff = coeff.copy()
281             perturbed_coeff[coeff_name] *= 1 + perturbation
282             forecast_df, _ = _simulate_forecast(base_emission, base_values, perturbed_coeff, assumptions)

```

```

283     peak = _summarize_peak(forecast_df, scenario_order)
284     for scenario in scenario_order:
285         result = peak.set_index("情景").loc[scenario]
286         base = baseline.loc[scenario]
287         rows.append(
288             {
289                 "情景": scenario,
290                 "扰动系数": coeff_name,
291                 "系数说明": label,
292                 "扰动比例": perturbation,
293                 "达峰年份": result["达峰年份"],
294                 "达峰年份变化": result["达峰年份"] - base["达峰年份"],
295                 "峰值_Mt": result["峰值_Mt"],
296                 "峰值变化_Mt": result["峰值_Mt"] - base["峰值_Mt"],
297                 "2045年_Mt": result["2045年_Mt"],
298                 "2045变化_Mt": result["2045年_Mt"] - base["2045年_Mt"],
299                 "2045较峰值下降比例": result["2045较峰值下降比例"],
300             }
301         )
302     return pd.DataFrame(rows)
303
304
305 def run_problem_three(
306     carbon: pd.DataFrame,
307     national: pd.DataFrame,
308     province: pd.DataFrame,
309     problem_two: dict[str, pd.DataFrame] | None = None,
310 ) -> dict[str, pd.DataFrame]:
311     total = carbon[carbon["Sector"].eq("Total")].copy()
312     annual = total.groupby("Year", as_index=False)["CO2 (Mt)"].sum()
313     days = total.groupby("Year")["Date"].nunique().reset_index(name="记录天数")
314     annual = annual.merge(days, on="Year", how="left")
315     annual["是否完整年份"] = annual["记录天数"] >= 365
316     base_year = 2024
317     if base_year not in annual["Year"].values:
318         raise ValueError("附件1缺少2024年Sector=Total数据,无法确定预测基准。")
319     base_emission = float(annual.loc[annual["Year"].eq(base_year), "CO2 (Mt)"].iloc[0])
320     base_values, base_table = _prepare_national_base(national, province, base_year)
321     if problem_two is None:
322         problem_two = run_problem_two(province)
323     coeff = problem_two["ols_coef"].set_index("变量")["系数"].to_dict()
324     required = ["lnP", "lnA", "煤炭相关排放占比", "第二产业占比", "城镇化率"]
325     missing = [name for name in required if name not in coeff]
326     if missing:
327         raise ValueError(f"问题二 OLS 系数缺少变量: {missing}")
328
329     coeff_rows = pd.DataFrame(
330         [{"变量": key, "STIRPAT_OLS系数": coeff[key], "预测中含义": "驱动变量变化对 ln(CO2) 的边际影响"} for key in required]
331     )
332     assumptions = scenario_driver_assumptions()
333     scenario_order = list(assumptions.keys())
334     forecast_df, params_df = _simulate_forecast(base_emission, base_values, coeff, assumptions, base_year)
335     peak_df = _summarize_peak(forecast_df, scenario_order, base_year)
336     single_factor_sensitivity = _single_factor_sensitivity(base_emission, base_values, coeff, assumptions, peak_df)
337     coefficient_sensitivity = _coefficient_sensitivity(base_emission, base_values, coeff, assumptions, peak_df)
338     sector_2024 = (
339         carbon[(carbon["Year"].eq(2024)) & (~carbon["Sector"].eq("Total"))]
340         .groupby("Sector", as_index=False)["CO2 (Mt)"]
341         .sum()
342         .sort_values("CO2 (Mt)", ascending=False)
343     )
344
345     RESULT_DIR.mkdir(parents=True, exist_ok=True)
346     write_csv(annual, RESULT_DIR / "01_全国年度排放.csv")
347     write_csv(sector_2024, RESULT_DIR / "02_2024部门排放.csv")
348     write_csv(params_df, RESULT_DIR / "03_情景参数.csv")
349     write_csv(forecast_df, RESULT_DIR / "04_2024-2045预测序列.csv")
350     write_csv(peak_df, RESULT_DIR / "05_达峰与减排潜力.csv")
351     write_csv(coeff_rows, RESULT_DIR / "06_STIRPAT 递推系数.csv")
352     write_csv(base_table, RESULT_DIR / "07_驱动变量基准值.csv")
353     write_csv(single_factor_sensitivity, RESULT_DIR / "08_单因素灵敏度分析.csv")
354     write_csv(coefficient_sensitivity, RESULT_DIR / "09_STIRPAT 系数灵敏度分析.csv")
355
356     return {
357         "annual": annual,
358         "sector_2024": sector_2024,
359         "params": params_df,
360         "forecast": forecast_df,
361         "peak": peak_df,
362         "stirpat_coef": coeff_rows,
363         "driver_base": base_table,
364         "single_factor_sensitivity": single_factor_sensitivity,
365         "coefficient_sensitivity": coefficient_sensitivity,
366     }
367
368
369 def make_problem_three_figures(problem_three: dict[str, pd.DataFrame]) -> None:
370     forecast = problem_three["forecast"]
371     peak = problem_three["peak"].copy()
372     scenario_order = ["基准情景", "低碳情景", "强化低碳情景"]
373     fig, ax = plt.subplots(figsize=(13, 5.5))
374     plot_data = forecast[forecast["年份"].between(2024, 2045)].rename(columns={"预测 CO2_Mt": "预测 CO2 排放量 (Mt)"})

```

```

375 sns.lineplot(
376     data=plot_data,
377     x=" 年份",
378     y=" 预测 CO2 排放量 (Mt) ",
379     hue=" 情景",
380     hue_order=scenario_order,
381     linewidth=2.4,
382     marker="o",
383     markersize=3,
384     ax=ax,
385 )
386 color_map = {text.get_text(): handle.get_color() for text, handle in zip(ax.legend_.texts, ax.legend_.legend_handles)}
387 y_min = plot_data[" 预测 CO2 排放量 (Mt) "].min() * 0.97
388 ax.set_ylim(y_min, plot_data[" 预测 CO2 排放量 (Mt) "].max() * 1.03)
389 peak[" 情景"] = pd.Categorical(peak[" 情景"], categories=scenario_order, ordered=True)
390 peak = peak.sort_values(" 情景").reset_index(drop=True)
391 peak[" 情景"] = peak[" 情景"].astype(str)
392 for _, row in peak.iterrows():
393     color = color_map.get(row[" 情景"], "#555555")
394     ax.vlines(row[" 达峰年份"], y_min, row[" 峰值 _Mt"], color=color, linestyle="--", linewidth=1.2, alpha=0.75)
395     ax.scatter(row[" 达峰年份"], row[" 峰值 _Mt"], color=color, edgecolor="white", linewidth=0.8, s=58, zorder=5)
396 peak_year_colors = {int(row[" 达峰年份"]): color_map.get(row[" 情景"], "#555555") for _, row in peak.iterrows()}
397 ax.set_xticks([2024, 2025, 2026, 2030, 2032, 2035, 2040, 2045])
398 ax.tick_params(axis="x", labelsz=10, pad=6)
399 for label in ax.get_xticklabels():
400     year = int(float(label.get_text()))
401     if year in peak_year_colors:
402         label.set_color(peak_year_colors[year])
403 ax.legend(title=" 情景", loc="upper left", bbox_to_anchor=(1.01, 1), frameon=True)
404 ax.set_title("2024-2045 年全国 CO2 排放三情景 STIRPAT 预测")
405 ax.set_xlabel(" 年份")
406 ax.set_ylabel("CO2 排放量 (Mt) ")
407 save_fig(fig, FIG_DIR, "07_ 三情景碳排放趋势图")
408
409 peak = problem_three["peak"].copy()
410 peak[" 情景"] = pd.Categorical(peak[" 情景"], categories=scenario_order, ordered=True)
411 peak = peak.sort_values(" 情景").reset_index(drop=True)
412 peak[" 情景"] = peak[" 情景"].astype(str)
413 fig, ax1 = plt.subplots(figsize=(8.5, 5.5))
414 scenario_palette = {" 基准情景": "#4C72B0", " 低碳情景": "#DD8452", " 强化低碳情景": "#55A868"}
415 sns.barplot(data=peak, x="情景", y="峰值_Mt", hue="情景", order=scenario_order, ax=ax1, palette=scenario_palette,
416             legend=False)
417 for patch in ax1.patches:
418     current_width = patch.get_width()
419     new_width = current_width * 0.72
420     patch.set_x(patch.get_x() + (current_width - new_width) / 2)
421     patch.set_width(new_width)
422 ax1.set_title(" 三情景峰值水平与 2045 减排潜力")
423 ax1.set_ylabel(" 峰值排放 (Mt) ")
424 ax1.set_xlabel("")
425 left_ticks = [10000, 10600, 11200, 11800, 12400, 13000]
426 right_ticks = [0, 9, 18, 27, 36, 45]
427 ax1.set_ylim(10000, 13000)
428 ax1.set_yticks(left_ticks)
429 ax1.grid(True, axis="y")
430 ax2 = ax1.twinx()
431 line_color = "#4F46E5"
432 ax2.set_ylim(0, 45)
433 ax2.set_yticks(right_ticks)
434 ax2.grid(False)
435 ax2.plot(peak[" 情景"], peak["2045 较峰值下降比例"] * 100, color=line_color, marker="o", linewidth=2.2)
436 ax2.set_ylabel("2045 较峰值下降比例 (%) ")
437 for i, row in peak.reset_index().iterrows():
438     ax1.text(i, row[" 峰值 _Mt"], f'{row[" 达峰年份"]}', ha="center", va="bottom", fontsize=10)
439     ratio = row["2045 较峰值下降比例"] * 100
440     offset = 1.8 if i != 2 else -2.2
441     ax2.text(
442         i,
443         ratio + offset,
444         f'{row["2045 较峰值下降比例"]:.1%}',
445         ha="center",
446         va="bottom" if offset > 0 else "top",
447         color="#312E81",
448         fontsize=10,
449         bbox={"boxstyle": "round,pad=0.2", "facecolor": "white", "edgecolor": "none", "alpha": 0.82},
450     )
451 save_fig(fig, FIG_DIR, "08_ 三情景峰值与减排潜力对比图")
452
453 single = problem_three["single_factor_sensitivity"].copy()
454 single_plot = single[(single[" 情景"].eq(" 基准情景")) & (single[" 扰动比例"].isin([-0.20, 0.20]))].copy()
455 single_plot[" 扰动方向"] = single_plot[" 扰动比例"].map({-0.20: "-20%", 0.20: "+20%"})
456 order = (
457     single_plot.groupby(" 扰动变量")["2045 变化 _Mt"]
458     .apply(lambda s: s.abs().max())
459     .sort_values(ascending=True)
460     .index.tolist()
461 )
462 fig, ax = plt.subplots(figsize=(8.8, 5.4))
463 sns.barplot(
464     data=single_plot,
465     y=" 扰动变量",
466     x="2045 变化 _Mt",

```

```

466     hue=" 扰动方向",
467     order=order,
468     palette={"-20%": "#60A5FA", "+20%": "#F97316"},
469     ax=ax,
470 )
471 ax.axvline(0, color="#333333", linewidth=1)
472 ax.set_title(" 单因素灵敏度: 基准情景 2045 排放影响")
473 ax.set_xlabel("2045 年 CO2 排放变化量 (Mt) ")
474 ax.set_ylabel("")
475 ax.legend(title=" 扰动比例", frameon=True)
476 save_fig(fig, FIG_DIR, "09_ 单因素灵敏度 _2045 排放影响图")
477
478 coef = problem_three["coefficient_sensitivity"].copy()
479 coef_plot = coef[(coef[" 情景"].eq(" 基准情景")) & (coef[" 扰动比例"].isin([-0.10, 0.10]))].copy()
480 coef_plot[" 扰动方向"] = coef_plot[" 扰动比例"].map({-0.10: "-10%", 0.10: "+10%"})
481 order = (
482     coef_plot.groupby(" 系数说明")["2045 变化 _Mt"]
483     .apply(lambda s: s.abs().max())
484     .sort_values(ascending=True)
485     .index.tolist()
486 )
487 fig, ax = plt.subplots(figsize=(8.8, 5.4))
488 sns.barplot(
489     data=coef_plot,
490     y=" 系数说明",
491     x="2045 变化 _Mt",
492     hue=" 扰动方向",
493     order=order,
494     palette={"-10%": "#93C5FD", "+10%": "#8B5CF6"},
495     ax=ax,
496 )
497 ax.axvline(0, color="#333333", linewidth=1)
498 ax.set_title("STIRPAT 系数灵敏度: 基准情景 2045 排放影响")
499 ax.set_xlabel("2045 年 CO2 排放变化量 (Mt) ")
500 ax.set_ylabel("")
501 ax.legend(title=" 扰动比例", frameon=True)
502 save_fig(fig, FIG_DIR, "10_STIRPAT 系数灵敏度 _2045 排放影响图")
503
504
505 def main() -> None:
506     setup_style()
507     province, national, carbon = read_data()
508     problem_two = run_problem_two(province)
509     result = run_problem_three(carbon, national, province, problem_two)
510     make_problem_three_figures(result)
511     print(f" 问题三结果已生成: {RESULT_DIR}")
512     print(f" 问题三图片已生成: {FIG_DIR}")
513
514
515 if __name__ == "__main__":
516     main()

```

附录 F 问题四求解脚本 solve_problem4.py

```

1  from __future__ import annotations
2
3  """
4  问题四脚本说明
5
6  算法与模型:
7  - 不再重新建模, 而是读取问题一的省份分类、问题二的驱动因素检验、问题三的情景预测结果。
8  - 按“区域分类-关键驱动-达峰预测”逻辑生成政策建议报告。
9
10  使用数据:
11  - 输入: 问题一、问题二、问题三脚本返回的结果字典, 或由 run_all.py 统一传入。
12  - 输出: 问题四政策建议报告文本。
13  """
14
15  import sys
16  from pathlib import Path
17
18  CODE_ROOT = Path(__file__).resolve().parents[1]
19  ROOT = CODE_ROOT.parent
20  sys.pycache_prefix = str(ROOT / ".pycache")
21  sys.path.append(str(CODE_ROOT))
22
23  import pandas as pd
24
25  from common import PROBLEM_RESULT_DIRS, read_data, setup_style
26  from 问题一.solve_problem1 import run_problem_one
27  from 问题二.solve_problem2 import run_problem_two
28  from 问题三.solve_problem3 import run_problem_three
29
30
31  PROBLEM = " 问题四"
32  RESULT_DIR = PROBLEM_RESULT_DIRS[PROBLEM]
33
34
35  def write_policy_report(problem_one: dict[str, pd.DataFrame], problem_two: dict[str, pd.DataFrame], problem_three: dict[str,
↵ pd.DataFrame]) -> str:

```

```

36 cluster = problem_one["cluster"]
37 ridge = problem_two["ridge_coef"].sort_values(" 绝对值排序")
38 peak = problem_three["peak"].set_index(" 情景")
39 resource = cluster[cluster[" 类型"].eq(" 资源依赖高排放型")][" 省份"].tolist()
40 industrial = cluster[cluster[" 类型"].eq(" 工业制造高排放型")][" 省份"].tolist()
41 main_drivers = ", ".join(ridge.head(2)[" 变量"].tolist())
42
43 policy = f""" 问题四：政策建议报告
44
45 基于省际分类、STIRPAT驱动因素识别和三情景预测结果，我国推进“双碳”目标应坚持“总量控制、结构优化、效率提升、区域分类”的思路。首先，对{"、
↳ ".join(resource)}等资源依赖高排放型省份，应将煤炭消费总量控制和煤电低碳改造作为重点，推动煤炭清洁高效利用、煤电灵活性改造、
↳ 新能源基地建设和资源型产业转型，避免经济增长继续依赖高碳能源扩张。其次，对{"、".join(industrial)}等工业制造高排放型省份，应聚焦钢铁、
↳ 建材、电力、石化和交通等重点行业，实施节能改造、余热利用、电气化替代和绿色供应链管理，建立单位产品碳强度约束。再次，
↳ 中等转型压力型省份应在承接产业转移时设置能耗和碳排放准入门槛，防止高耗能产业简单转移，同时提高能源利用效率和非化石能源占比。北京、
↳ 上海等经济发达效率型地区应发挥技术、金融和治理优势，推广绿色金融、碳交易、数字化碳管理和低碳消费。模型结果表明，
↳ {main_drivers}是最稳定的关键驱动因素；情景预测显示，基准情景约在{int(peak.loc["基准情景", "达峰年份"])}年达峰，
↳ 强化低碳情景下2045年较峰值下降{peak.loc["强化低碳情景", "2045较峰值下降比例"]:.1%}。因此，应持续降低煤炭占比，
↳ 扩大风光水核等非化石能源供给，强化技术创新和碳市场机制，形成差异化、可考核、可持续的减排路径。
"""
46
47 RESULT_DIR.mkdir(parents=True, exist_ok=True)
48 (RESULT_DIR / " 问题四 _ 政策建议报告.txt").write_text(policy, encoding="utf-8")
49 return policy
50
51
52 def main() -> None:
53     setup_style()
54     province, national, carbon = read_data()
55     problem_one = run_problem_one(province)
56     problem_two = run_problem_two(province)
57     problem_three = run_problem_three(carbon, national, province, problem_two)
58     write_policy_report(problem_one, problem_two, problem_three)
59     print(f" 问题四报告已生成：{RESULT_DIR}")
60
61
62 if __name__ == "__main__":
63     main()

```